EXAMINATION SYSTEM FOR VISUALLY IMPAIRED

(Automation-Based Examination and Evaluation)

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in

COMPUTER SCIENCE & ENGINEERING

MEDICAPS UNIVERSITY

Session: Jan-Jun 2026

By

Mayank Mitra

Mishika Kasliwal

Section - 6K

Under the Guidance of

Prof. Garima Tukra

Dr. Sheetal Bawane

Department of Computer Science & Engineering

Report Approval

The project work "EXAMINATION SYSTEM FOR VISUALLY IMPAIRED (AI-Based Examination and Evaluation)" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner
Name : Prof. Garima Tukra
Designation: Assistant Proffessor
Affiliation : Medicaps University

Internal Examiner
Name : Dr. Sheetal Bawane
Designation: Assistant Proffessor
Affiliation : Medicaps University

Declaration

We hereby declare that the project entitled "EXAMINATION SYSTEM FOR VISUALLY IMPAIRED (AI-Based Examination and Evaluation)" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of <Name, designation and department of the Guide(s)>, Faculty of Engineering, Medi-Caps University Indore, is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and names of the students with date

_____ _____

Mayank Mitra Mishika Kasliwal

Certificate

We, Prof. Garima Tukra and Sheetal Bawane, certify that the project entitled "EXAMINATION SYSTEM FOR VISUALLY IMPAIRED (Automation-Based Examination and Evaluation)" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Mayank Mitra and Mishika Kasliwal is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Garima Tukra (CSE Department)

Medi-Caps University, Indore

Prof. Sheetal Bawane (CSE Department)

Medi-Caps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank Prof. (Dr.) Pramod S. Nair, Dean, Faculty of Engineering, Medi-Caps University, for giving us the opportunity to work on this project. We would also like to thank our Head of the Department Dr. Ratnesh Litoriya for his continuous encouragement for the betterment of the project.

We extend our sincere thanks to our project guide for providing valuable guidance, insights, and constant motivation throughout the development of this project. Their expertise in the relevant domains helped us shape the project into a meaningful and practical solution.

It is their help and support due to which we were able to complete the design and technical report. Without their support this report would not have been possible.

Mayank Mitra **Mishika Kasliwal**

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

The Examination System for Visually Impaired (EASVI) is an AI-powered, web-based platform designed to provide an inclusive and accessible examination experience for visually impaired students. Traditional online examination systems are heavily dependent on visual interaction and mouse-based navigation, making them inherently inaccessible to individuals with visual disabilities. EASVI addresses this critical gap by integrating voice-based interaction, keyboard-only navigation, and AI-generated questions using the Gemini model.

The system is built using HTML, TailwindCSS, and JavaScript on the frontend, with Django as the backend framework. The Web Speech API provides text-to-speech and speech synthesis capabilities, enabling the system to announce all actions and read out questions to the user. The Gemini API is used to dynamically generate unique examination questions, reducing the possibility of exam fraud and improving the quality of assessments. Results are evaluated instantly upon submission and announced to the user via voice output, eliminating delays common in traditional systems.

The proposed system includes a fully accessible user interface with pages for login, a student dashboard, exam instructions, the examination itself, and a results page. All pages are navigable through keyboard shortcuts, ensuring no reliance on mouse interaction. The project demonstrates a significant improvement over existing assistive examination solutions such as JAWS screen reader-based systems, ATutor, and ProctorTrack, which lack AI question generation, have limited automation, and offer poor voice integration.

EASVI provides a self-reliant, equitable, and technologically advanced examination platform that empowers visually impaired students to demonstrate their true academic potential without dependence on scribes or manual assistance. The system contributes meaningfully to the goal of inclusive education and digital accessibility.

Keywords: Visually Impaired, Accessibility, Web Speech API, Gemini AI, Inclusive Examination, Voice Instructions, Django, Keyboard Navigation, Text-to-Speech

Table of Contents

	Report Approval	i
	Declaration	ii
	Certificate	iii
	Acknowledgement	iv
	Abstract	v
	Table of Contents	vi
Chapter 1	**Introduction**	
1.1	Introduction	
1.2	Literature Review	
1.3	Objectives	
1.4	Significance	
1.5	Problem Statement	
1.6	Proposed Solution	
Chapter 2	**Requirements Specification**	
2.1	User Characteristics	
2.2	Functional Requirements	
2.3	Dependencies	
2.4	Performance Requirements	
2.5	Hardware Requirements	
2.6	Constraints & Assumptions	
Chapter 3	**System Design**	
Chapter 4	**Implementation, Testing, and Maintenance**	
Chapter 5	**Results and Discussions**	
Chapter 6	**Summary and Conclusions**	
Chapter 7	**Future Scope**	
	Bibliography	

Chapter 1
INTRODUCTION

1.1 Introduction

Examinations are a cornerstone of the academic process, serving as the primary means of evaluating students' knowledge, skills, and academic progress. However, despite tremendous advancements in educational technology, traditional examination systems continue to present significant barriers for students with visual impairments. These barriers range from inaccessible question paper formats and reliance on human scribes to the absence of voice-guided interfaces and assistive technologies that meet the unique needs of visually impaired users.

The rapid growth of the internet and web technologies has opened new avenues for creating inclusive digital platforms. Yet, most online examination systems are designed with a purely visual interface, requiring candidates to use a mouse, read text on screen, and interpret graphical elements—tasks that are inherently inaccessible to those who are blind or have severe visual impairment. This digital exclusion not only limits their academic participation but also contributes to broader social inequalities.

The Examination System for Visually Impaired (EASVI) is developed to bridge this accessibility gap. It is a fully web-based, AI-powered examination system that enables visually impaired students to take examinations independently, without any reliance on scribes or external human assistance. The system employs voice-based interaction through the Web Speech API, allowing text-to-speech synthesis to read out all questions, instructions, and announcements. Navigation is accomplished entirely through keyboard shortcuts, ensuring that no mouse interaction is required at any stage of the examination process.

A defining feature of EASVI is its integration with the Gemini AI model for automatic question generation. This ensures that each examination session presents a unique set of questions, thereby enhancing the academic integrity of the process and reducing the risk of question leakage. The backend, powered by the Django framework, manages user

authentication, session control, question retrieval from a MySQL or PostgreSQL database, and result computation. Results are evaluated instantly upon submission and communicated to the student via voice announcement, making the entire cycle from question presentation to result delivery accessible and efficient.

1.2 Literature Review

Several works and systems have been developed to address the challenges faced by visually impaired individuals in educational settings. A review of the existing literature and tools reveals both the progress made and the significant gaps that remain.

JAWS (Job Access With Speech) is one of the most widely used screen reading software tools that converts on-screen text into synthesized speech. While it enables visually impaired users to interact with computers in general, it is not specifically designed for online examination systems and requires significant configuration to work with standard exam platforms, many of which are not JAWS-compatible.

ATutor is an open-source Learning Management System (LMS) designed with accessibility in mind. It supports keyboard navigation and screen reader compatibility to a reasonable degree. However, it is primarily a learning platform and lacks dedicated examination-specific functionality such as timed assessments, dynamic question generation, and instant result announcement.

ProctorTrack is a remote proctoring solution that includes some support for screen readers and keyboard navigation. However, it remains a supplementary tool rather than an end-to-end accessible examination system. Its primary focus is invigilation rather than accessibility, and it does not provide voice-guided question delivery or AI-generated questions.

Research studies have highlighted that the majority of online examination documents are not compatible with screen readers, and that many platforms require significant manual modification to be usable by visually impaired students. Furthermore, existing systems do

not incorporate AI-based question generation, which limits their ability to provide personalized or varied assessments.

The reviewed literature confirms that while some efforts have been made towards accessible education and examination, a comprehensive, AI-integrated, voice-first examination system tailored specifically for visually impaired students is largely absent from the current landscape. EASVI is designed to fill this gap.

1.3 Objectives

The primary objectives of the EASVI project are as follows:

- To design and develop a fully accessible, web-based examination system for visually impaired students that eliminates dependence on scribes and visual interaction.
- To implement voice-based interaction using the Web Speech API, enabling text-to-speech synthesis for all examination content including questions, instructions, and results.
- To integrate the Gemini AI model for dynamic and automatic generation of examination questions, ensuring uniqueness and fairness in assessments.
- To ensure complete keyboard-only navigation throughout the system, removing all requirements for mouse-based input.
- To provide instant automated result evaluation and voice-based result announcement upon submission of the examination.
- To build a fully accessible user interface comprising a login page, dashboard, instructions page, examination page, and results page.
- To support print functionality for examination results to allow students to obtain a physical record of their performance.

1.4 Significance

The development of EASVI holds considerable significance from both a social and technological perspective. From a social standpoint, visually impaired students represent an underserved demographic in the context of digital education. Their inability to access standard online examination platforms places them at a severe disadvantage and undermines the principles of inclusive education. EASVI directly addresses this inequity by providing a platform that is designed from the ground up to be accessible, empowering, and self-reliant.

From a technological perspective, EASVI represents a meaningful integration of multiple cutting-edge technologies—the Web Speech API, the Gemini AI model, the Django web framework, and relational database management—into a cohesive and functional system. The use of AI for question generation is particularly significant, as it moves beyond static question banks and introduces a dynamic, scalable approach to examination content creation.

Furthermore, EASVI aligns with broader global initiatives such as the United Nations Sustainable Development Goal 4 (Quality Education) and the Web Content Accessibility Guidelines (WCAG), which advocate for inclusive and equitable education for all individuals regardless of physical ability. The system can serve as a model for future accessible examination platforms in both academic institutions and professional certification bodies.

1.5 Problem Statement

Current online examination systems have the following significant limitations with respect to visually impaired users:

- Heavy dependence on visual interaction, making them inaccessible to individuals who are blind or have severe visual impairments.
- Limited or no voice support, requiring users to rely on external screen reader software that may not be compatible with the examination platform.

- Requirement of mouse navigation, which is impossible for many visually impaired users without additional assistive devices.
- Use of static question banks where the same set of questions is repeated across examination sessions, increasing the risk of fraud and reducing assessment reliability.
- Delays in result processing, which create uncertainty and anxiety for students who cannot read results on their own.
- Interfaces not optimized for visually impaired accessibility, with poor contrast, unlabeled form elements, and missing ARIA attributes.

Research has confirmed that many online examination documents are not compatible with screen readers and require extensive manual modification before they can be used by visually impaired students. This situation is neither scalable nor sustainable in a modern educational environment that increasingly relies on digital assessments.

1.6 Proposed Solution

EASVI proposes a comprehensive, voice-first, AI-integrated examination system that addresses each of the identified limitations. The key components of the proposed solution are:

- Voice Announcements: All system actions, including question reading, navigation cues, and result announcements, are delivered through synthesized speech using the Web Speech API, removing all dependence on visual output.
- Keyboard-Only Navigation: The entire examination process is navigable using standard keyboard shortcuts, with no requirement for mouse interaction at any stage.
- AI-Powered Question Generation: Integration with the Gemini AI model enables the automatic and dynamic generation of examination questions, ensuring each student receives a unique set of questions per session.
- Instant Result Evaluation: Upon submission, the system immediately computes and announces the result, eliminating delays and manual intervention.

- Print Support: Students can print their results directly from the system, providing a physical record of their performance.
- Accessible UI: All interface components are built with full accessibility compliance, including ARIA attributes, high-contrast design, and logical tab order, ensuring compatibility with assistive technologies.

Chapter 2
REQUIREMENTS SPECIFICATION

2.1 User Characteristics

The primary users of the EASVI system are visually impaired students who require an accessible, voice-guided interface to participate in online examinations. These users may have varying degrees of visual impairment ranging from low vision to complete blindness. They typically rely on keyboard navigation and audio feedback to interact with digital systems. Secondary users include examination administrators and faculty members who may use the system to configure examinations, monitor activity, and review results. These users are expected to have standard computer literacy and may interact with the system through a conventional visual interface.

2.2 Functional Requirements

The functional requirements of the EASVI system are as follows:

- The system shall provide a secure login mechanism for students, authenticating their credentials against the database.
- The system shall present a dashboard upon successful login, summarizing available examinations and profile information.
- The system shall read all instructions aloud to the student using the Web Speech API before the examination begins.
- The system shall generate a unique set of examination questions using the Gemini API for each examination session.
- The system shall read each question and all available answer options aloud using synthesized speech.
- The system shall allow the student to navigate between questions and select answers using keyboard shortcuts only.
- The system shall evaluate answers instantly upon submission and compute the final score.

- The system shall announce the result aloud, including the score and pass/fail status.
- The system shall allow the student to print the result page for physical documentation.
- The system shall support administrative functions for managing user accounts and examination configuration.

2.3 Dependencies

The EASVI system has the following external dependencies:

- Web Speech API: Available in modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) to provide text-to-speech and speech synthesis capabilities.
- Gemini API: Google's Gemini language model API, accessed via an authenticated API key, for dynamic question generation.
- Django Framework: Python-based web framework for building and managing the backend server, routes, and application logic.
- MySQL / PostgreSQL: Relational database management system for storing user credentials, question banks, session data, and examination results.
- TailwindCSS: Utility-first CSS framework for building responsive and accessible front-end layouts.
- Internet Connection: Required for Gemini API communication and general web application access.

2.4 Performance Requirements

The system should meet the following performance benchmarks to ensure a seamless examination experience:

- Question generation via the Gemini API should complete within 3 seconds under standard internet conditions.
- Speech synthesis for each question should begin within 1 second of the question being loaded.

- Result computation and announcement should occur within 2 seconds of examination submission.
- The system should support concurrent sessions for multiple students without degradation in performance.
- Database read/write operations for student data and results should complete within 500 milliseconds.

2.5 Hardware Requirements

The following hardware is required to run the EASVI system:

Component	Specification
Processor	Intel Core i3 or equivalent (1 GHz minimum)
RAM	Minimum 4 GB
Storage	500 MB free disk space for server-side installation
Audio Output	Speakers or headphones (required for voice output)
Network	Stable internet connection (broadband recommended)

2.6 Constraints & Assumptions

The following constraints and assumptions apply to the EASVI system:

- The system assumes that the user's web browser supports the Web Speech API. While most modern browsers do, some older browsers may not, and compatibility should be verified before deployment.
- The Gemini API requires a valid API key and an active internet connection. Any disruption to the internet connection during question generation may result in a failure to load the examination.
- The system is designed for web-based deployment and does not provide a native mobile application in the current version.
- It is assumed that the examination administrator will correctly configure the examination parameters, including the subject, duration, and number of questions, before the examination session is initiated.

- The system does not currently support audio input (speech-to-answer) and requires keyboard input for answer selection.

Chapter 3
SYSTEM DESIGN

3.1 System Architecture

EASVI follows a client-server architecture where the front-end interface runs in the student's web browser, and the back-end server handles business logic, AI integration, and database management. The overall system is composed of four major layers: the Presentation Layer (HTML, TailwindCSS, JavaScript), the Accessibility Layer (Web Speech API), the Application Layer (Django Framework), and the Data Layer (MySQL / PostgreSQL).

The interaction flow within the system can be described as follows: the user accesses the web interface through a browser, interacts using keyboard inputs, and receives feedback via synthesized speech. All user requests are routed through the Django backend, which communicates with the Gemini API for question generation and with the database for data persistence. Results are returned to the front-end and announced via voice output.

3.2 Data Flow

The data flow within the system follows this sequence:

1. The user navigates to the login page and enters credentials via keyboard.
2. The Django backend authenticates the user against the database and initiates a session.
3. Upon successful login, the user is directed to the dashboard, where available examinations are listed.
4. When the user initiates an examination, the Django backend sends a request to the Gemini API with the examination subject and parameters.
5. The Gemini API responds with a dynamically generated set of questions, which are stored temporarily in the session.
6. Questions are delivered to the front-end one at a time, with the Web Speech API reading each question and its options aloud.

7. The user selects an answer using keyboard input, and the selection is recorded in the session.
8. Upon submission, the Django backend evaluates the answers, computes the score, and stores the result in the database.
9. The result page is rendered, and the Web Speech API announces the final score and performance status.

3.3 System Design Components

3.3.1 Activity Diagram

The activity diagram of EASVI illustrates the sequential flow of actions from student login to result announcement. The student logs in, views the dashboard, reads instructions, begins the examination, answers questions one by one using keyboard navigation, submits the examination, and receives the result via voice announcement. Error handling flows for incorrect login credentials and API unavailability are also incorporated.

3.3.2 Flow Chart

The system flowchart begins with the user accessing the EASVI URL. The system checks whether the user is authenticated. If not, the user is directed to the login page. Upon successful authentication, the dashboard is displayed. The student selects an examination, after which questions are fetched from the Gemini API. Questions are presented one by one with voice output. The student answers each question and, upon completion, submits the examination. The system computes the result, stores it in the database, and announces it via voice.

3.3.3 Class Diagram

The primary classes in the EASVI system include: User (attributes: userId, name, email, passwordHash; methods: login(), logout()), Examination (attributes: examId, subject, duration, questionCount; methods: generateQuestions(), submitExam()), Question (attributes: questionId, questionText, optionsList, correctAnswer), and Result (attributes:

resultId, userId, examId, score, timestamp; methods: computeResult(), announceResult(), printResult()).

3.3.4 Sequence Diagram

The sequence diagram for EASVI depicts the interactions between the Student, the Web Interface, the Django Backend, the Gemini API, and the Database. The sequence begins with the student sending a login request, followed by the backend validating credentials with the database. Upon successful login, an examination request triggers the backend to call the Gemini API, which returns generated questions. The interface presents questions to the student, captures responses, and, upon submission, the backend computes and stores the result before sending it back to the interface for voice announcement.

3.4 Database Design

3.4.1 Logical Database Design

The logical database design of EASVI includes the following primary entities and their relationships:

- Users Table: Stores student and administrator credentials including user ID, name, email, hashed password, and role.
- Examinations Table: Contains examination metadata including exam ID, subject, duration, number of questions, and creation timestamp.
- Questions Table: Stores generated question text, associated options, and the correct answer, linked to a specific examination session.
- Results Table: Records the outcome of each examination session including the student ID, exam ID, score achieved, total marks, and timestamp.

3.4.2 Physical Database Design

The physical database is implemented using MySQL or PostgreSQL. Foreign key constraints are used to maintain referential integrity between the Users, Examinations, and Results tables. Indexes are created on frequently queried fields such as userId and examId to optimize query performance. Sessions are managed server-side by Django using its built-in session middleware, with session data stored in the database.

Chapter 4
IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies

The EASVI system is implemented using the following technologies:

Technology	Component	Purpose
HTML5	Frontend	Structure and markup of web pages
TailwindCSS	Frontend	Responsive, accessible UI styling
JavaScript (ES6+)	Frontend	Dynamic interaction and API calls
Web Speech API	Accessibility Layer	Text-to-speech and voice synthesis
Python 3.x	Backend	Server-side programming language
Django 4.x	Backend	Web framework and routing
Gemini API	AI Integration	Dynamic question generation
MySQL / PostgreSQL	Database	Data persistence and retrieval
VS Code	IDE	Development environment
Postman	Testing Tool	API endpoint testing

4.2 Testing Techniques and Test Plans

The EASVI system is tested using the following techniques:

- Unit Testing: Individual components such as the login module, question generation API call, and result computation function are tested in isolation using Python's unittest framework and Django's built-in test utilities.
- Integration Testing: Tests are conducted to verify the correct interaction between the frontend, Django backend, Gemini API, and the database. This includes end-to-end tests for the complete examination flow from login to result.
- Accessibility Testing: The system is tested using screen reader simulation tools to ensure all voice announcements are accurate and all keyboard navigation paths are functional. ARIA attributes are validated using browser developer tools.

- User Acceptance Testing (UAT): Simulated testing with users representing visually impaired students is conducted to gather feedback on the usability and accessibility of the system.
- Performance Testing: Load testing is performed to evaluate system response time under concurrent user sessions.

4.3 Installation Instructions

The following steps describe how to install and run the EASVI application on a local server:

10. Ensure that Python 3.x and pip are installed on the system.
11. Clone or download the project repository to the local machine.
12. Navigate to the project directory and create a virtual environment: python -m venv env
13. Activate the virtual environment: source env/bin/activate (Linux/Mac) or env\Scripts\activate (Windows)
14. Install required Python packages: pip install -r requirements.txt
15. Configure the database settings in settings.py with the appropriate MySQL or PostgreSQL credentials.
16. Run database migrations: python manage.py migrate
17. Set the Gemini API key in the environment variables or the project configuration file.
18. Start the Django development server: python manage.py runserver
19. Open a Web Speech API-compatible browser and navigate to http://localhost:8000

4.4 End User Instructions

The following instructions guide visually impaired students in using the EASVI system:

20. Open a compatible web browser (Google Chrome recommended for best Web Speech API support).
21. Navigate to the EASVI URL as provided by the institution.

22. On the Login Page, use the Tab key to move between fields and enter your username and password. Press Enter to submit.
23. On the Dashboard, use the Tab key to select an available examination and press Enter to begin.
24. Listen carefully to the voice instructions on the Instructions Page before the examination begins.
25. During the examination, questions and options will be read aloud automatically. Use the arrow keys or Tab to navigate between options and press Enter to select an answer.
26. Use keyboard shortcuts to move to the next question. At the end of the examination, press Enter on the Submit button.
27. The result will be announced automatically. Use the keyboard to activate the Print button if a physical copy is required.

Chapter 5
RESULTS AND DISCUSSIONS

5.1 User Interface Representation

The EASVI system comprises five primary user-facing pages, each designed with full accessibility compliance. These are the Login Page, the Dashboard Page, the Instructions Page, the Examination Page, and the Results Page. The overall interface is built using HTML5 and TailwindCSS, ensuring a clean, high-contrast visual design that is also compatible with assistive technologies and screen readers.

5.2 Brief Description of Various Modules

The system is organized into the following functional modules:

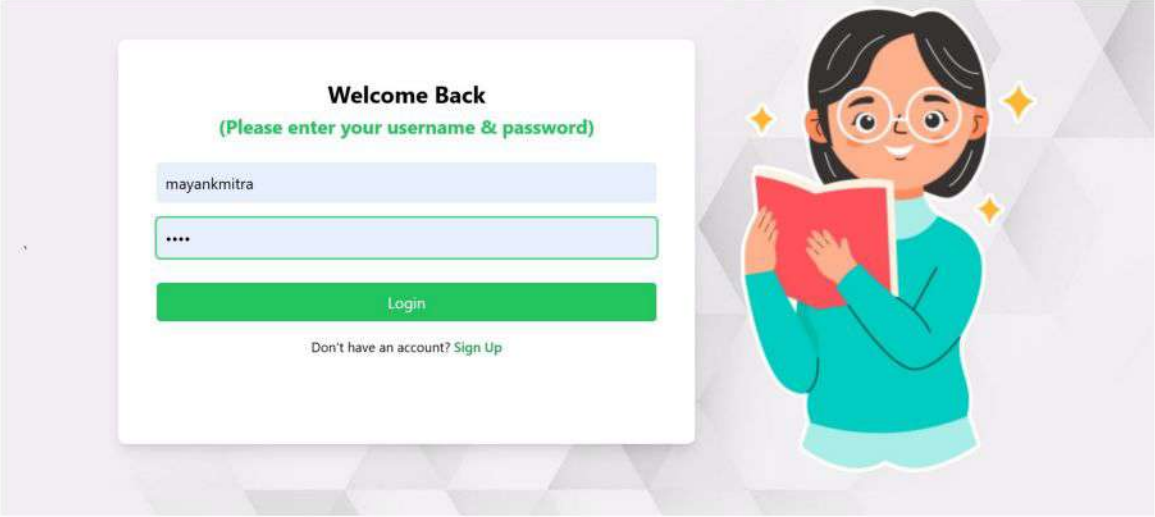
- Authentication Module: Manages user login and session creation. Validates credentials against the database and redirects authenticated users to the dashboard.
- Dashboard Module: Displays available examinations and user profile summary. Allows the student to initiate an examination session.
- Instructions Module: Presents pre-examination guidelines to the student via both text and voice output, ensuring the student is prepared before the examination begins.
- Examination Module: The core module that interfaces with the Gemini API to generate questions, renders them on screen, reads them aloud, and captures the student's keyboard-based answers.
- Result Module: Evaluates submitted answers, computes the score, stores the result in the database, announces it via voice, and provides a print option.

5.3 Snapshots of System

The following pages constitute the user interface of EASVI:

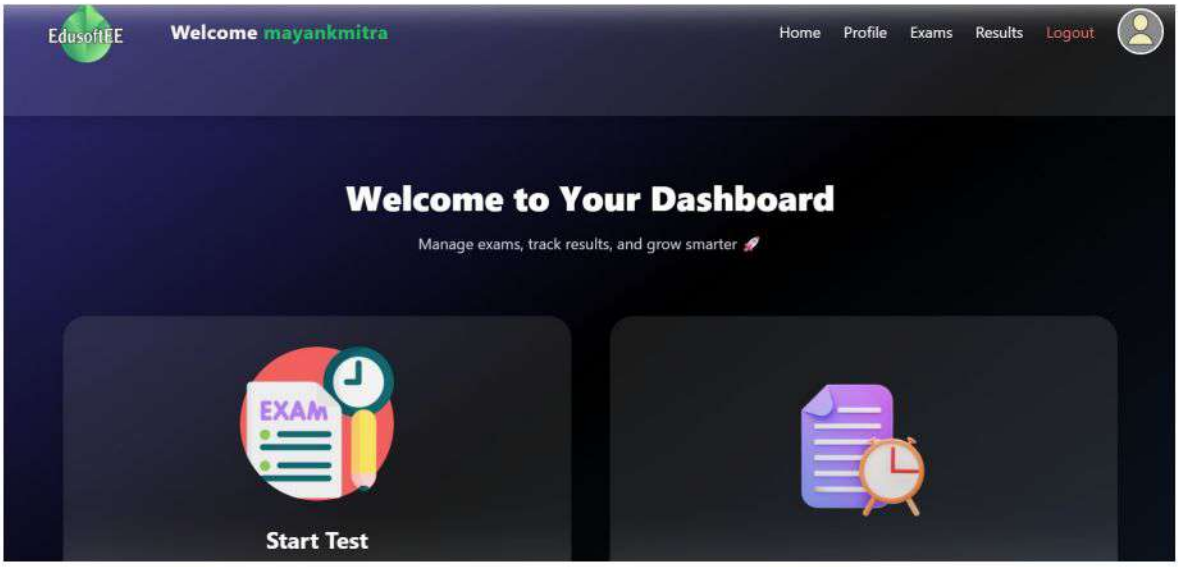
Login Page:

The Login Page presents a clean, minimal interface with clearly labeled username and password fields and a login button. The Web Speech API announces the page name and prompts the user to enter their credentials upon page load.



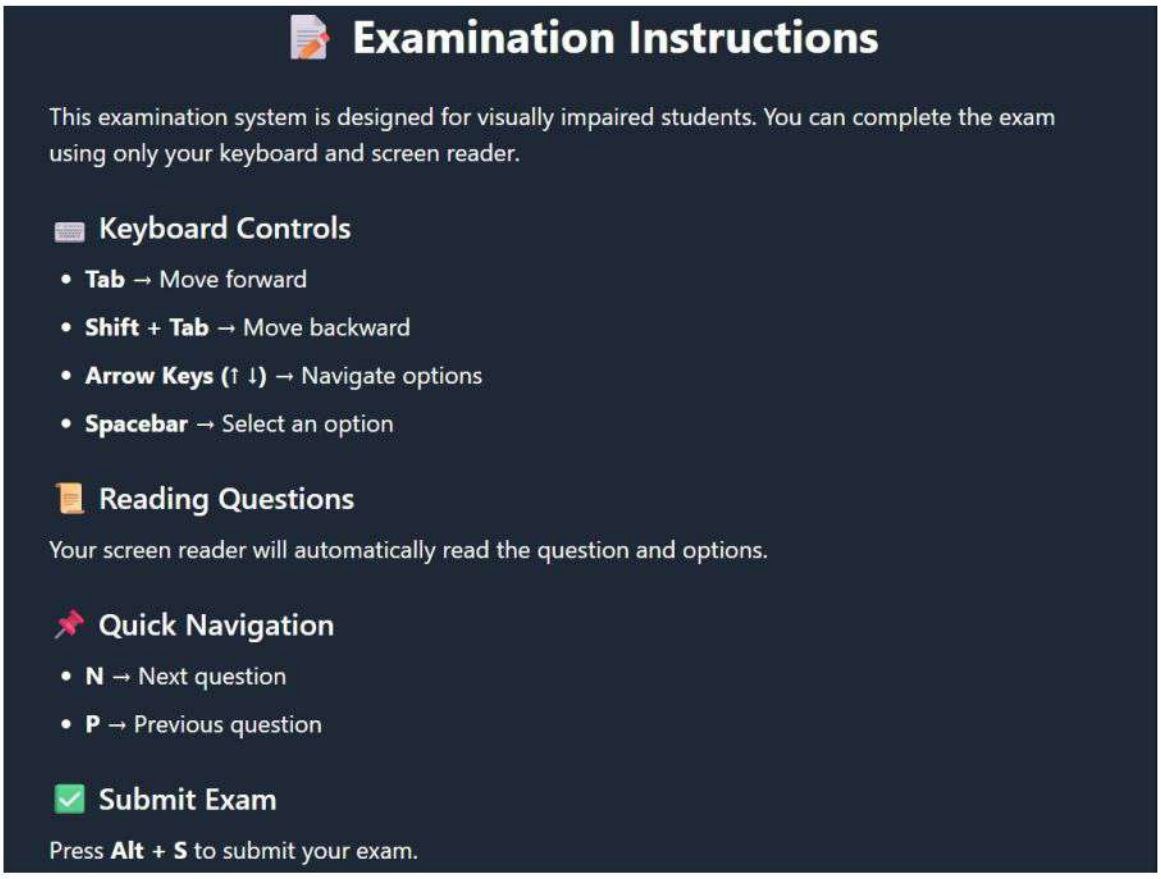
Dashboard Page:

The Dashboard Page greets the student by name using synthesized speech and lists available examinations as accessible buttons. Navigation between examination options is accomplished using the Tab key. A brief description of each examination, including subject and duration, is announced when the student focuses on each option.



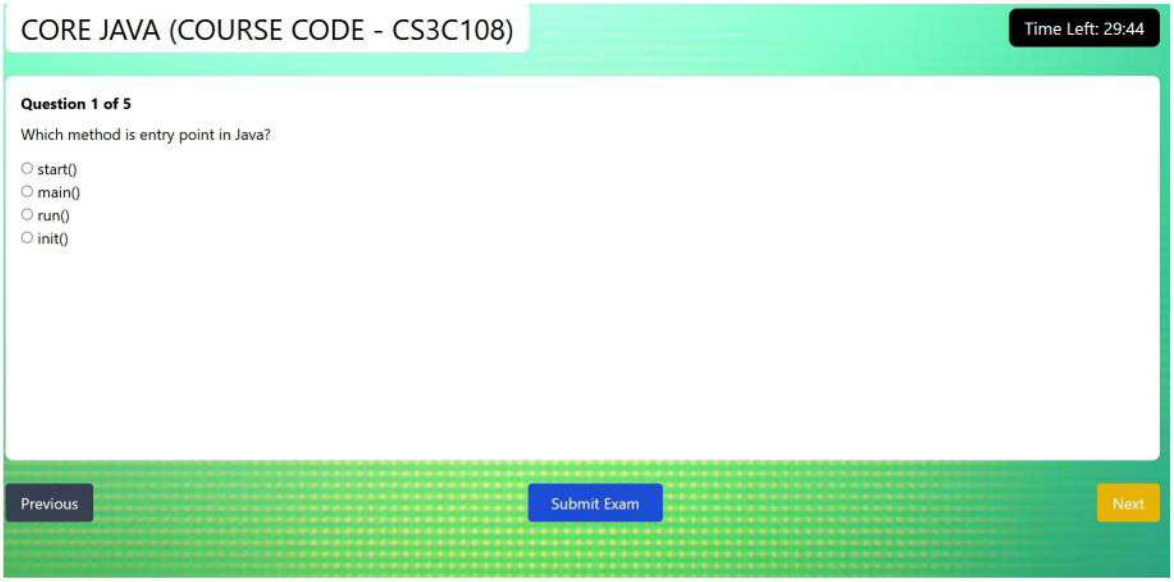
Instructions Page:

The Instructions Page displays and reads aloud the examination rules and guidelines. It informs the student of keyboard shortcuts, the examination duration, the total number of questions, and the marking scheme. A "Begin Examination" button is available for the student to proceed when ready.



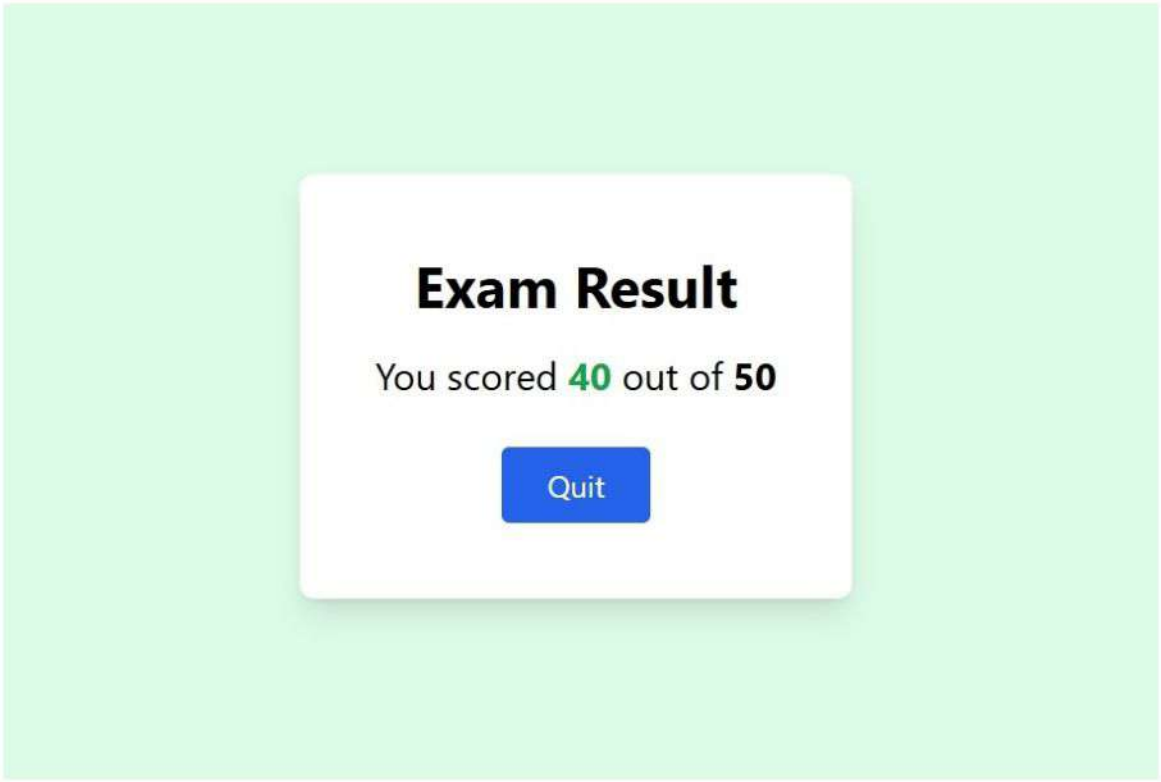
Examination Page:

The Examination Page is the central module of the system. Each question is displayed on screen and read aloud along with its answer options. A timer is displayed and optionally announced at regular intervals. The student navigates between options using keyboard keys and confirms their selection before moving to the next question. The page includes a progress indicator and a Submit button that is available after the last question is answered.



Result Page:

The Result Page displays the student's score, percentage, and pass/fail status in large, high-contrast text. The Web Speech API announces the result immediately upon page load. A Print button allows the student to obtain a physical copy of their result.



5.4 Back End Representation

The backend of EASVI is powered by the Django framework. The Django ORM manages all database interactions, abstracting raw SQL queries into Python objects. The backend exposes RESTful API endpoints for login, question retrieval, answer submission, and result retrieval. Gemini API calls are made server-side to prevent API key exposure on the client. Django's session middleware manages user authentication states across page navigation.

5.5 Database Tables

The key database tables in the EASVI system are as follows:

- users: Stores user credentials and roles (student/admin). Key fields include id, username, email, password_hash, and role.
- examinations: Contains examination metadata. Key fields include id, subject, duration_minutes, and question_count.
- results: Records examination outcomes. Key fields include id, user_id (foreign key), exam_id (foreign key), score, total_marks, and created_at.

Chapter 6
SUMMARY AND CONCLUSIONS

6.1 Summary

The Examination System for Visually Impaired (EASVI) is a fully accessible, AI-integrated, web-based examination platform designed specifically to meet the needs of visually impaired students. The project was conceived in response to the widespread inaccessibility of existing online examination systems, which rely heavily on visual interaction and mouse-based navigation, rendering them unusable for students with visual impairments.

The system was developed using a modern technology stack comprising HTML5, TailwindCSS, and JavaScript on the front-end, the Web Speech API for voice-based interaction, Django as the backend framework, the Gemini AI model for dynamic question generation, and MySQL/PostgreSQL for data management. The system provides a complete examination lifecycle—from login and dashboard access through question presentation and answer selection to result evaluation and announcement—all through keyboard navigation and voice feedback.

The project involved a thorough review of existing assistive examination systems including JAWS, ATutor, and ProctorTrack, identifying key gaps in their functionality with respect to AI-based question generation, voice integration, and examination-specific accessibility. EASVI directly addresses these gaps by delivering a purpose-built, voice-first solution.

6.2 Conclusions

The following conclusions are drawn from the development and evaluation of the EASVI system:

- EASVI successfully demonstrates that it is technically feasible to build a fully accessible online examination system using freely available web technologies, without requiring proprietary assistive software.

- The integration of the Web Speech API enables effective voice-based interaction, allowing visually impaired students to independently access all examination functions without assistance.
- The use of the Gemini AI model for question generation significantly enhances the fairness and academic integrity of the examination process by providing unique question sets for each session.
- The keyboard-only navigation design ensures that the system is fully operable without a mouse, meeting a fundamental accessibility requirement for visually impaired users.
- The instant result evaluation and voice announcement features eliminate the delays and manual dependencies associated with traditional result processing, improving the overall examination experience.
- EASVI represents a significant advancement over existing accessible examination solutions and can serve as a replicable model for institutions seeking to provide inclusive digital assessments.

Chapter 7
FUTURE SCOPE

The EASVI system, while comprehensive in its current form, presents several opportunities for enhancement and extension in future iterations:

- Speech-to-Answer Input: A natural next step is to incorporate speech recognition capabilities, allowing students to answer questions verbally rather than through keyboard input. This would further reduce the physical interaction requirement and improve the examination experience for students with motor disabilities in addition to visual impairments.
- Mobile Application: Developing a native mobile application for Android and iOS platforms would significantly expand the reach of EASVI, enabling students to take examinations on smartphones and tablets, which are increasingly the primary devices used by students in developing regions.
- Multi-Language Support: Adding support for multiple regional languages in both the question generation and voice output components would make EASVI accessible to a broader population of visually impaired students across India and internationally.
- Adaptive Assessment: Future versions could incorporate AI-driven adaptive assessment, where the difficulty of questions adjusts dynamically based on the student's performance during the examination, providing a more accurate measure of their knowledge.
- Proctoring Integration: Integration with accessible proctoring solutions that do not rely on visual surveillance could enhance the integrity of remote examinations for visually impaired students.
- Offline Functionality: Implementing offline examination capability using service workers and local storage would allow students in areas with poor internet connectivity to take examinations without disruption.
- Analytics Dashboard for Educators: A dedicated dashboard for faculty and administrators providing insights into student performance, common errors, and examination analytics would enhance the pedagogical value of the platform.

Bibliography

[1] W3C Web Accessibility Initiative, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[2] Google LLC, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/docs

[3] Mozilla Developer Network, "Web Speech API," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[4] Django Software Foundation, "Django Documentation," Django Project, 2024. [Online]. Available: https://docs.djangoproject.com/

[5] A. Yesilada, S. Harper, C. Goble, and R. Stevens, "Screen Readers Cannot See," in Proc. International Conference on Web Engineering, 2004, pp. 445–458.

[6] P. Menger, "Accessibility in Online Assessment: A Review of Current Practice and Future Directions," Journal of Inclusive Education Technology, vol. 12, no. 3, pp. 45–62, 2022.

[7] Freedom Scientific, "JAWS Screen Reader," 2024. [Online]. Available: https://www.freedomscientific.com/products/software/jaws/

[8] ATutor, "ATutor Learning Management System," 2024. [Online]. Available: https://atutor.github.io/

[9] Verificient Technologies, "ProctorTrack Remote Proctoring Solution," 2024. [Online]. Available: https://www.verificient.com/proctortrack/

[10] TailwindCSS, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

EXAMINATION SYSTEM FOR VISUALLY IMPAIRED

(AI-Based Examination and Evaluation)

A Synopsis for Minor Project

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Mayank Mitra
Mishika Kasliwal

Under the Guidance of
Prof. Garima Tukra
Dr. Sheetal Bawane

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331
April 2025

1. Introduction

Examinations are a cornerstone of the academic process, serving as the primary means of evaluating students' knowledge, skills, and academic progress. However, despite significant advancements in educational technology, traditional and online examination systems continue to present substantial barriers for students with visual impairments. These barriers include inaccessible question paper formats, reliance on human scribes, absence of voice-guided interfaces, and platforms built exclusively for mouse-based visual interaction.

The Examination System for Visually Impaired (EASVI) is developed to bridge this accessibility gap. It is a fully web-based, AI-powered examination system that enables visually impaired students to take examinations independently, without any reliance on scribes or external human assistance. The system employs voice-based interaction through the Web Speech API, keyboard-only navigation, and dynamic question generation using the Gemini AI model. The backend is powered by the Django framework with MySQL/PostgreSQL for data management. EASVI aligns with global inclusive education initiatives including the United Nations Sustainable Development Goal 4 (Quality Education) and the Web Content Accessibility Guidelines (WCAG) [1].

2. Literature Review

A review of existing assistive and accessible examination solutions reveals both progress and significant gaps:

- **JAWS (Job Access With Speech) [7]:** A widely used screen reading tool that converts on-screen text into synthesized speech. While it enables general computer interaction, it is not specifically designed for online examination systems and requires significant configuration to work with standard exam platforms, many of which are not JAWS-compatible.
- **ATutor [8]:** An open-source Learning Management System (LMS) designed with accessibility in mind. It supports keyboard navigation and screen reader compatibility, but lacks dedicated examination-specific functionality such as timed assessments, dynamic question generation, and instant result announcement.
- **ProctorTrack [9]:** A remote proctoring solution with limited screen reader and keyboard support. Its primary focus is invigilation rather than accessibility, and it provides neither voice-guided question delivery nor AI-generated questions.

Research studies have confirmed that the majority of online examination documents are not compatible with screen readers and require extensive manual modification before they can be used by visually impaired students [5][6]. A comprehensive, AI-integrated, voice-first examination system tailored specifically for visually impaired students is largely absent from the current landscape. EASVI is designed to fill this gap.

3. Problem Definition

Current online examination systems have the following significant limitations with respect to visually impaired users:

- Heavy dependence on visual interaction, making platforms inaccessible to individuals who are blind or have severe visual impairments.
- Limited or no integrated voice support, requiring users to rely on external screen reader software that may not be compatible with the examination platform.
- Mandatory mouse navigation, which is impossible for many visually impaired users without additional assistive devices.
- Use of static question banks where identical questions are repeated across examination sessions, increasing the risk of fraud and reducing assessment reliability.
- Significant delays in result processing, creating uncertainty for students who cannot independently read results.

- Interfaces not optimized for visually impaired accessibility — poor contrast, unlabeled form elements, and missing ARIA attributes.

4. Objectives

The primary objectives of the EASVI project are:

- To design and develop a fully accessible, web-based examination system for visually impaired students that eliminates dependence on scribes and visual interaction.
- To implement voice-based interaction using the Web Speech API, enabling text-to-speech synthesis for all examination content including questions, instructions, and results.
- To integrate the Gemini AI model for dynamic and automatic generation of examination questions, ensuring uniqueness and fairness in every assessment session [2][3].
- To ensure complete keyboard-only navigation throughout the system, removing all requirements for mouse-based input.
- To provide instant automated result evaluation and voice-based result announcement upon submission of the examination.
- To build a fully accessible user interface comprising a login page, dashboard, instructions page, examination page, and results page.
- To support print functionality for examination results so students can obtain a physical record of their performance.

5. Methodology

5.1 System Architecture

EASVI follows a client-server architecture composed of four major layers: the Presentation Layer (HTML5, TailwindCSS [10], JavaScript), the Accessibility Layer (Web Speech API [3]), the Application Layer (Django Framework [4]), and the Data Layer (MySQL/PostgreSQL). The user interacts through a browser using keyboard inputs and receives all feedback via synthesized speech. All requests are routed through the Django backend, which communicates with the Gemini API for question generation and with the database for data persistence.

5.2 Technology Stack

- Frontend: HTML5, TailwindCSS, JavaScript (ES6+)
- Accessibility Layer: Web Speech API (text-to-speech and speech synthesis)
- Backend: Python 3.x and Django 4.x web framework
- AI Integration: Google Gemini API for dynamic question generation

- Database: MySQL / PostgreSQL for user data, session management, and results
- Development Tools: VS Code (IDE), Postman (API testing)

5.3 System Workflow

The examination workflow proceeds as follows:

1. The student navigates to the EASVI login page and authenticates using keyboard-only input; the Web Speech API announces all prompts.

2. Upon login, the dashboard lists available examinations, with each option announced via synthesized speech when focused.

3. The instructions page reads all examination rules, keyboard shortcuts, duration, and marking scheme aloud before the session begins.

4. When the exam starts, the Django backend sends a request to the Gemini API, which returns a unique, dynamically generated set of questions for that session.

5. Questions and answer options are presented one at a time and read aloud. The student navigates using arrow keys / Tab and selects answers via keyboard.

6. On submission, the backend instantly evaluates answers, computes the score, stores the result in the database, and announces it via voice output.

7. The student may print the result page for a physical record of their performance.

5.4 System Modules

- **Authentication Module:** Manages user login and session creation. Validates credentials against the database and redirects authenticated users to the dashboard.
- **Dashboard Module:** Displays available examinations and user profile summary. Allows the student to initiate an examination session.
- **Instructions Module:** Delivers pre-examination guidelines to the student via both text and synthesized voice output.
- **Examination Module:** Core module that interfaces with the Gemini API to generate questions, renders them on screen, reads them aloud, and captures keyboard-based answers.
- **Result Module:** Evaluates submitted answers, computes the score, stores the result in the database, announces it via voice, and provides a print option.

5.5 Testing Approach

- Unit Testing: Individual components including the login module, question generation API call, and result computation are tested using Python's unittest framework and Django's test utilities.
- Integration Testing: End-to-end verification of interactions between the frontend, Django backend, Gemini API, and database.
- Accessibility Testing: Screen reader simulation and ARIA attribute validation ensure all voice announcements and keyboard navigation paths are functional.
- User Acceptance Testing (UAT): Simulated testing with users representing visually impaired students to gather usability feedback.
- Performance Testing: Load testing to evaluate system response time under concurrent user sessions.

References

[1]. W3C Web Accessibility Initiative, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[2]. Google LLC, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/docs

[3]. Mozilla Developer Network, "Web Speech API," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[4]. Django Software Foundation, "Django Documentation," Django Project, 2024. [Online]. Available: https://docs.djangoproject.com/

[5]. A. Yesilada, S. Harper, C. Goble, and R. Stevens, "Screen Readers Cannot See," in Proc. International Conference on Web Engineering, 2004, pp. 445–458.

[6]. P. Menger, "Accessibility in Online Assessment: A Review of Current Practice and Future Directions," Journal of Inclusive Education Technology, vol. 12, no. 3, pp. 45–62, 2022.

[7]. Freedom Scientific, "JAWS Screen Reader," 2024. [Online]. Available: https://www.freedomscientific.com/products/software/jaws/

[8]. ATutor, "ATutor Learning Management System," 2024. [Online]. Available: https://atutor.github.io/

[9]. Verificient Technologies, "ProctorTrack Remote Proctoring Solution," 2024. [Online]. Available: https://www.verificient.com/proctortrack/

[10]. TailwindCSS, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

Software Requirements Specifications(SRS)

Examination System for Visually Impaired (EASVI)

Mayank Mitra & Mishika Kasliwal

(EN23CS301604) (EN23CS301617)



Section : 6K

Session : Jan-Jun 2026

Under the Guidance of

Prof. Garima Tukra

Dr. Sheetal Bawane

Computer Science Engineering Department

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive, authoritative description of the EASVI (Examination System for Visually Impaired) system. It formally defines the functional requirements, non-functional requirements, system constraints, external interface specifications, and expected system behavior.

The document adheres to the IEEE Standard 830-1998 for Software Requirements Specifications and is intended to serve as the authoritative reference throughout the software development lifecycle — from design and implementation through testing and deployment.

This document is intended for the following stakeholders:

- Development Team — to understand the complete system requirements and implement accordingly
- Project Evaluators / Examiners — to verify that the system meets its specified objectives
- Faculty & Institutional Stakeholders — to validate that the system addresses real accessibility needs
- Quality Assurance Team — to design test cases based on specified functional and non-functional requirements
- Future Maintainers — to understand design decisions and system boundaries

1.2 Scope of the Project

EASVI is a web-based, AI-powered examination system designed exclusively for visually impaired students. The system addresses a critical accessibility gap in conventional examination infrastructure by eliminating dependency on human scribes, tactile Braille materials, or other external accommodations.

The system provides the following core capabilities:

- Voice-based interaction using the Web Speech API (text-to-speech and speech-to-text)
- Complete keyboard-only navigation for users unable to use pointing devices
- AI-driven, dynamically generated examination questions using the Gemini API
- Automated, instant answer evaluation without manual intervention
- Voice-announced result delivery with optional printable result output
- Secure user authentication with accessible login/signup flows
- Role-based access supporting both student and administrator personas

EASVI is explicitly scoped to the examination domain for visually impaired individuals. It does not cover general-purpose e-learning, content delivery, or any domain outside computer-administered testing.

1.3 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- Web Speech API Specification — W3C Community Group Report, https://wicg.github.io/speech-api/
- Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs
- Django Framework Documentation — https://docs.djangoproject.com/
- Web Content Accessibility Guidelines (WCAG) 2.1 — W3C Recommendation
- WAI-ARIA Authoring Practices 1.1 — W3C Working Group Note
- OWASP Web Application Security Testing Guide

1.4 Overview of Document

The remainder of this document is organized as follows:

- Section 2 — Overall Description: Product perspective, architecture overview, user characteristics, operating environment, constraints, and assumptions.
- Section 3 — System Features (Functional Requirements): Detailed specification of all system functions with inputs, processing logic, and expected outputs.
- Section 4 — External Interface Requirements: User interface, hardware interface, software interface, and communication interface specifications.
- Section 5 — Non-Functional Requirements: Performance, accessibility, security, reliability, usability, and scalability requirements.
- Section 6 — System Models: UML diagrams including Use Case, Class, Activity, Component, Deployment, and State diagrams.
- Section 7 — Data Requirements: Data entities, database schema design, and data dictionary.
- Section 8 — Appendices: UI screen descriptions, future enhancements, and known limitations.

2. OVERALL DESCRIPTION

2.1 Product Perspective

EASVI is a standalone, web-based application developed to serve as an independent examination platform for visually impaired students. It is not a plugin or module of any existing Learning Management System (LMS), though future integration with institutional LMS platforms is identified as a potential enhancement.

The system operates as a three-tier web application: a browser-rendered frontend accessed by students, a Django-powered application server handling business logic, and a relational database storing persistent data. The system integrates two critical external services — the Google Gemini AI API for question generation and the browser-native Web Speech API for all voice interactions.

System Architecture Overview:

Layer	Technology	Responsibility
Presentation	HTML5, TailwindCSS, JavaScript	Accessible UI rendering, keyboard navigation, voice control
Application	Django (Python 3.x)	Business logic, authentication, exam management, evaluation
Data	MySQL / PostgreSQL	Persistent storage of users, exams, questions, results
AI Integration	Google Gemini API	Dynamic question generation based on configured topics
Accessibility	Web Speech API (browser-native)	Text-to-speech output and speech-to-text input

2.2 Product Functions

At a high level, EASVI performs the following primary functions. Each is elaborated in detail in Section 3.

- F1 — User Authentication: Secure login and registration with voice-guided feedback.
- F2 — Voice Instruction System: Automated, contextual voice guidance for all system interactions.
- F3 — Exam Initialization: Configuration and initiation of an examination session.
- F4 — AI-Based Question Generation: On-demand generation of unique, non-repeated questions via the Gemini API.

- F5 — Interactive Question Navigation: Keyboard-only navigation through questions with voice reading.
- F6 — Answer Recording: Capturing student responses via keyboard input or voice recognition.
- F7 — Automatic Evaluation: Real-time scoring of submitted answers by the backend.
- F8 — Voice Result Announcement: Audible delivery of examination scores and performance summary.
- F9 — Result Printing: Generation of a formatted, printable result document.
- F10 — Admin Management: Administrative functions for user management, exam configuration, and result review.

2.3 User Classes and Characteristics

User Class	Description	Technical Proficiency	Primary Interactions
Visually Impaired Student	Primary user; may have partial or total visual impairment	Basic computer literacy; familiar with keyboard navigation and screen readers	Login, exam navigation, answer submission, result retrieval
Administrator / Teacher	Manages system configuration, user accounts, and result reporting	Moderate technical proficiency; familiar with web-based dashboards	User management, exam topic configuration, result export
System Administrator	Responsible for server deployment, database management, and API key configuration	High technical proficiency; DevOps and backend familiarity required	Deployment, configuration, monitoring, backup

2.4 Operating Environment

Client-side Requirements:

- Operating System: Windows 10/11, macOS 10.15+, Ubuntu 20.04+ (or any OS with a modern browser)
- Browser: Google Chrome 90+, Mozilla Firefox 88+, Microsoft Edge 90+ (with Web Speech API support)
- Network: Stable internet connection with minimum 2 Mbps bandwidth
- Hardware: Standard keyboard; audio output device (speakers or headphones) mandatory
- Screen Reader (Optional): NVDA, JAWS, or VoiceOver may be used alongside the built-in voice system

Server-side Requirements:

- Server OS: Ubuntu 20.04 LTS or equivalent Linux distribution
- Runtime: Python 3.9+ with Django 4.x
- Database: MySQL 8.0+ or PostgreSQL 14+
- Memory: Minimum 4 GB RAM; 8 GB recommended for production
- Storage: Minimum 20 GB available disk space
- Outbound Connectivity: HTTPS access to Google Gemini API endpoints

2.5 Assumptions and Dependencies

Assumptions:

- All target users (visually impaired students) possess basic familiarity with keyboard navigation on a computer.
- Users have access to a functional audio output device and will operate in an environment where audio playback is feasible.
- Examination administrators will configure exam topics and question counts prior to student sessions.
- The system will be deployed in an environment with consistent internet access.

Dependencies:

- Google Gemini API — Any deprecation, quota change, or service interruption on Google's side will directly affect question generation functionality.
- Web Speech API — Functionality is browser-dependent and subject to browser vendor implementation differences.
- Django ORM — All data access is mediated through Django's ORM; raw SQL queries are not used.
- TailwindCSS CDN — The frontend relies on TailwindCSS for styling; a CDN-hosted version is assumed for prototype, with local build recommended for production.

3. SYSTEM FEATURES — FUNCTIONAL REQUIREMENTS

This section specifies all functional requirements of the EASVI system. Each feature is described using a structured use-case format that includes actors, preconditions, main flow, postconditions, and exception handling. Requirements are identified with unique IDs for traceability.

3.1 User Authentication (Login / Signup)

The authentication module is the entry point to the system. It must support both student and administrator personas, provide voice-guided instructions throughout, and enforce secure credential handling.

UC-01 — User Login

Attribute	Details
Use Case ID	UC-01
Use Case Name	User Login
Actor(s)	Visually Impaired Student, Admin
Preconditions	User has a registered account. System is accessible via a supported browser.
Trigger	User navigates to the EASVI URL and the login page is rendered.
Main Flow	1. System announces: 'Welcome to EASVI. Please enter your username and press Tab, then enter your password and press Enter to login.'2. User enters username and password using keyboard.3. System validates credentials against the database (bcrypt hash comparison).4. On success: System announces 'Login successful. Welcome, [Name].' and redirects to dashboard.5. On failure: System announces 'Invalid credentials. Please try again.' and clears the password field.
Postconditions	User session is established. Session token issued. Dashboard is loaded and announced via voice.
Exceptions / Alt Flows	E1 — Invalid credentials: Voice announces error; login attempt logged; account locked after 5 failed attempts.E2 — Network failure: Voice announces 'Unable to connect. Please check your internet connection.'E3 — Server error: Voice announces generic error; error logged server-side.

UC-02 — User Registration (Signup)

Attribute	Details

Use Case ID	UC-02
Use Case Name	User Registration (Signup)
Actor(s)	New Visually Impaired Student
Preconditions	User is not yet registered in the system.
Trigger	User navigates to the signup page (linked from login page; announced via voice).
Main Flow	1. System announces registration form fields in sequence.2. User enters: Full Name, Username, Email, Password, Confirm Password.3. System validates: username uniqueness, email format, password strength (min 8 characters, mixed case).4. On success: Account created, voice announces 'Registration successful. You may now log in.'5. System redirects to login page.
Postconditions	New user account persisted in the database. User can subsequently log in.
Exceptions / Alt Flows	E1 — Username already taken: Voice announces 'This username is already in use. Please choose another.'E2 — Password mismatch: Voice announces 'Passwords do not match. Please re-enter.'E3 — Invalid email format: Voice announces validation error.

3.2 Voice Interaction System

The voice interaction system is the accessibility backbone of EASVI. Every system action, page transition, form field, question, and result must be communicated to the user via synthesized speech. This module governs all TTS and STT interactions.

Functional Requirements:

- FR-V01: The system shall convert all question text, instructions, feedback messages, and result data to audible speech using the Web Speech API SpeechSynthesis interface.
- FR-V02: The system shall allow users to adjust voice speed (rate) and volume through keyboard-accessible settings, with changes taking immediate effect.
- FR-V03: The system shall support speech recognition (STT) as an optional answer input method, using the Web Speech API SpeechRecognition interface.
- FR-V04: All voice announcements shall be queued and played sequentially; simultaneous overlapping speech is not permitted.
- FR-V05: The system shall provide a keyboard shortcut (e.g., Spacebar or defined key) to pause and resume voice playback.
- FR-V06: The system shall re-read the current question or page content on demand (e.g., pressing the 'R' key).
- FR-V07: Voice feedback must be triggered within 300 milliseconds of any user action.

3.3 Exam Initialization

UC-03 — Initialize and Start Examination

Attribute	Details
Use Case ID	UC-03
Use Case Name	Initialize and Start Examination
Actor(s)	Visually Impaired Student
Preconditions	User is authenticated. Exam configuration (topics, question count, duration) has been set by an administrator.
Trigger	User selects 'Start Exam' from the dashboard (keyboard: Tab to button, Enter to activate).
Main Flow	1. System announces exam configuration: topic, number of questions, and time limit.2. System reads a full set of exam navigation instructions.3. User confirms readiness by pressing Enter.4. System sends request to Django backend, which calls the Gemini API to generate questions.5. Questions are received, validated (uniqueness, format check), and stored in the session.6. First question is rendered and read aloud.7. Exam timer begins.
Postconditions	Exam session is active. ExamSession record created in database with start_time. First question displayed and announced.
Exceptions / Alt Flows	E1 — Gemini API unavailable: Voice announces 'Unable to load questions. Please try again later.' Session is not created.E2 — Insufficient questions returned: System retries up to 3 times before aborting.E3 — Session timeout during initialization: User is returned to dashboard with voice error.

3.4 AI-Based Question Generation

Questions are generated on-demand using the Gemini AI API. The system sends a structured prompt specifying the exam topic, desired difficulty level, and question count. Generated questions are parsed and validated before being presented to students.

Functional Requirements:

- FR-Q01: The system shall request the configured number of multiple-choice questions from the Gemini API for each new exam session.
- FR-Q02: Each question shall include: question text, four answer options (labeled A–D), and the correct answer identifier.
- FR-Q03: The system shall validate that generated questions are well-formed before use. Malformed questions shall be re-requested.
- FR-Q04: The system shall ensure no question is repeated within a single exam session.

- FR-Q05: The Gemini API prompt shall specify the topic, difficulty level (easy, medium, hard as configured), and response format (JSON).
- FR-Q06: The system shall cache generated questions for the duration of the session; questions shall not be re-fetched during navigation.
- FR-Q07: The Gemini API key shall be stored as a server-side environment variable and shall not be exposed to the client.

3.5 Answer Submission and Navigation

UC-04 — Answer Questions and Navigate Exam

Attribute	Details
Use Case ID	UC-04
Use Case Name	Answer Questions and Navigate Exam
Actor(s)	Visually Impaired Student
Preconditions	Exam session is active. At least one question is loaded.
Trigger	Question is displayed and read aloud by the voice system.
Main Flow	1. Voice reads: '[Question Number] of [Total]. [Question text]. Option A: [text]. Option B: [text]. Option C: [text]. Option D: [text].'2. User selects an answer using keyboard keys (A, B, C, D) or arrow keys.3. System voice-confirms the selected option: 'You have selected Option [X]: [text]. Press Enter to confirm or press another letter to change.'4. User confirms selection (Enter) or changes selection.5. System saves the answer to the ExamSession.6. User navigates to next question (keyboard: N for next, P for previous).7. Previously answered questions show the saved response when revisited.
Postconditions	All answers are stored in the ExamSession. User can submit when all questions are answered (or when timer expires).
Exceptions / Alt Flows	E1 — No answer selected on navigation: Voice warns 'You have not answered this question. Press N to skip or select an option.'E2 — Timer expires: System auto-submits all recorded answers and announces 'Time is up. Your exam has been submitted.'

3.6 Automatic Evaluation System

Upon exam submission, the Django backend evaluates all recorded answers against the correct answers stored during question generation. Evaluation is performed server-side to prevent client-side manipulation.

Functional Requirements:

- FR-E01: The system shall compare each submitted answer against the corresponding correct answer retrieved from the session data.
- FR-E02: The system shall calculate the raw score (number of correct answers) and percentage score.
- FR-E03: Evaluation shall be completed and results persisted within 2 seconds of submission.
- FR-E04: Partial credit is not applicable in this version; each question is scored as fully correct or incorrect.
- FR-E05: The evaluation engine shall be stateless — it must produce identical results for identical inputs, regardless of the order of execution.
- FR-E06: Evaluated results shall be persisted to the Result entity in the database immediately after calculation.

4.3 Software Interfaces

System	Interface Type	Purpose	Error Handling
Google Gemini API	REST API (HTTPS/JSON)	Generate exam questions based on configured topic and difficulty	Retry up to 3 times; fallback to cached questions if available; abort session with voice error
Web Speech API	Browser-native JavaScript API	Text-to-speech output and optional speech-to-text input	Detect API availability on load; disable voice features with visual fallback if unavailable
Django ORM	Internal Python interface	All database read/write operations; no raw SQL permitted	ORM exceptions caught and logged; user-facing generic error messages
MySQL / PostgreSQL	SQL (via Django ORM)	Persistent storage of all system data	Connection pooling; automatic reconnect on transient failure
TailwindCSS	CSS framework (CDN or local build)	Responsive, accessible UI styling	Local fallback CSS if CDN unavailable

5. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and operational constraints that the system must satisfy. These requirements are testable and verifiable.

ID	Requirement	Specification
NFR-P01	Page Load Time	All pages shall load and be ready for interaction within 3 seconds on a 10 Mbps connection.
NFR-P02	Voice Response Latency	Voice feedback shall begin playing within 300 milliseconds of any user action.
NFR-P03	API Response Time	The Gemini API response for question generation shall complete within 5 seconds; if exceeded, the user is notified via voice.
NFR-P04	Evaluation Time	Exam evaluation and result generation shall complete within 2 seconds of submission.
NFR-P05	Concurrent Users	The system shall support a minimum of 100 concurrent exam sessions without performance degradation.
NFR-P06	Database Query Time	All database queries shall complete within 500 milliseconds under normal load.

6. SYSTEM MODELS

This section presents the UML models that describe the structural and behavioral characteristics of the EASVI system. Together, these diagrams provide a comprehensive view of the system from multiple perspectives.

6.1 Use Case Diagram

The Use Case Diagram identifies all actors interacting with EASVI and the system functionalities each actor can perform.

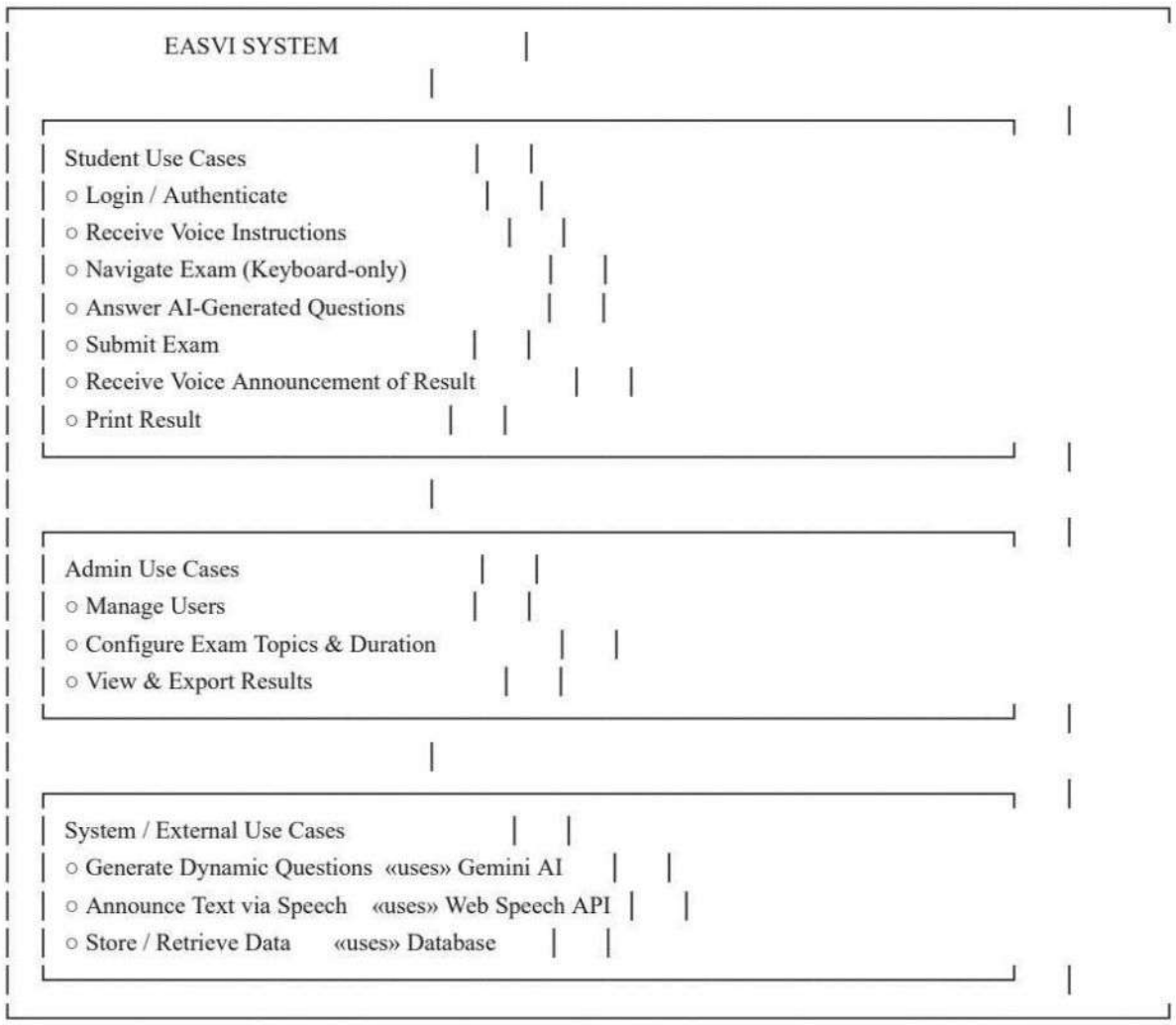
Actors

Actors in EASVI

Actor	Role
Visually Impaired Student	Primary actor — takes exam, navigates via keyboard/voice
Admin / Teacher	Creates exams, reviews results, manages users

Gemini AI API	External system — generates dynamic questions
Web Speech API	External system — converts text to speech and vice-versa
Database (MySQL/PostgreSQL)	External system — stores user data, questions, results

Use Cases



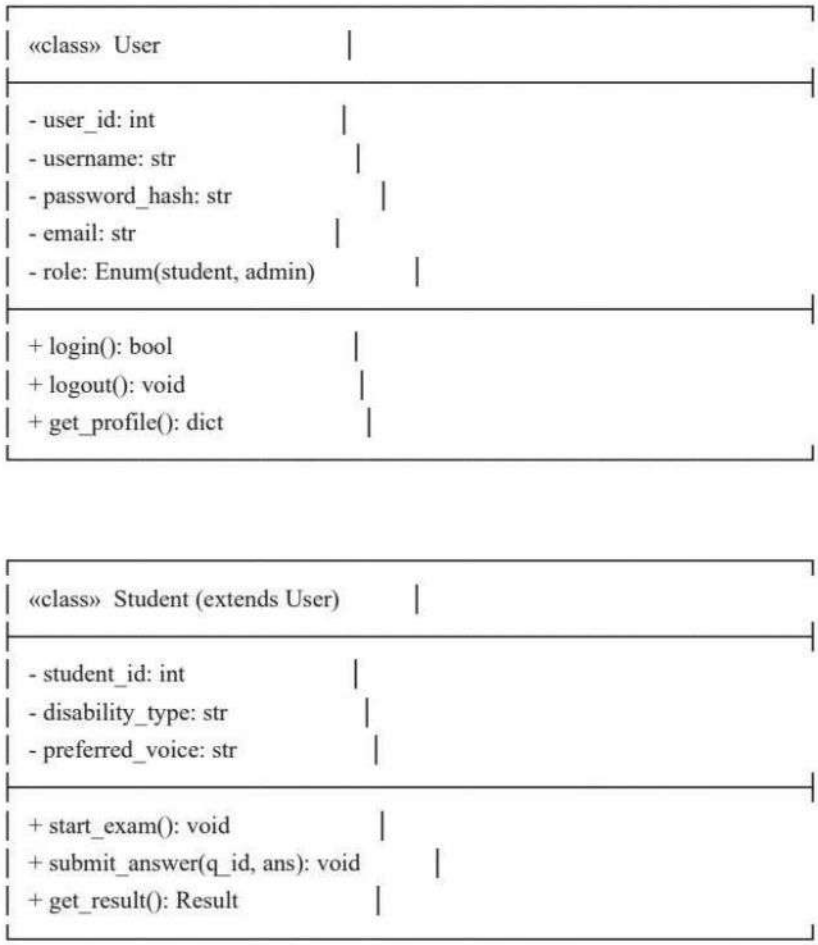
Use Case Descriptions

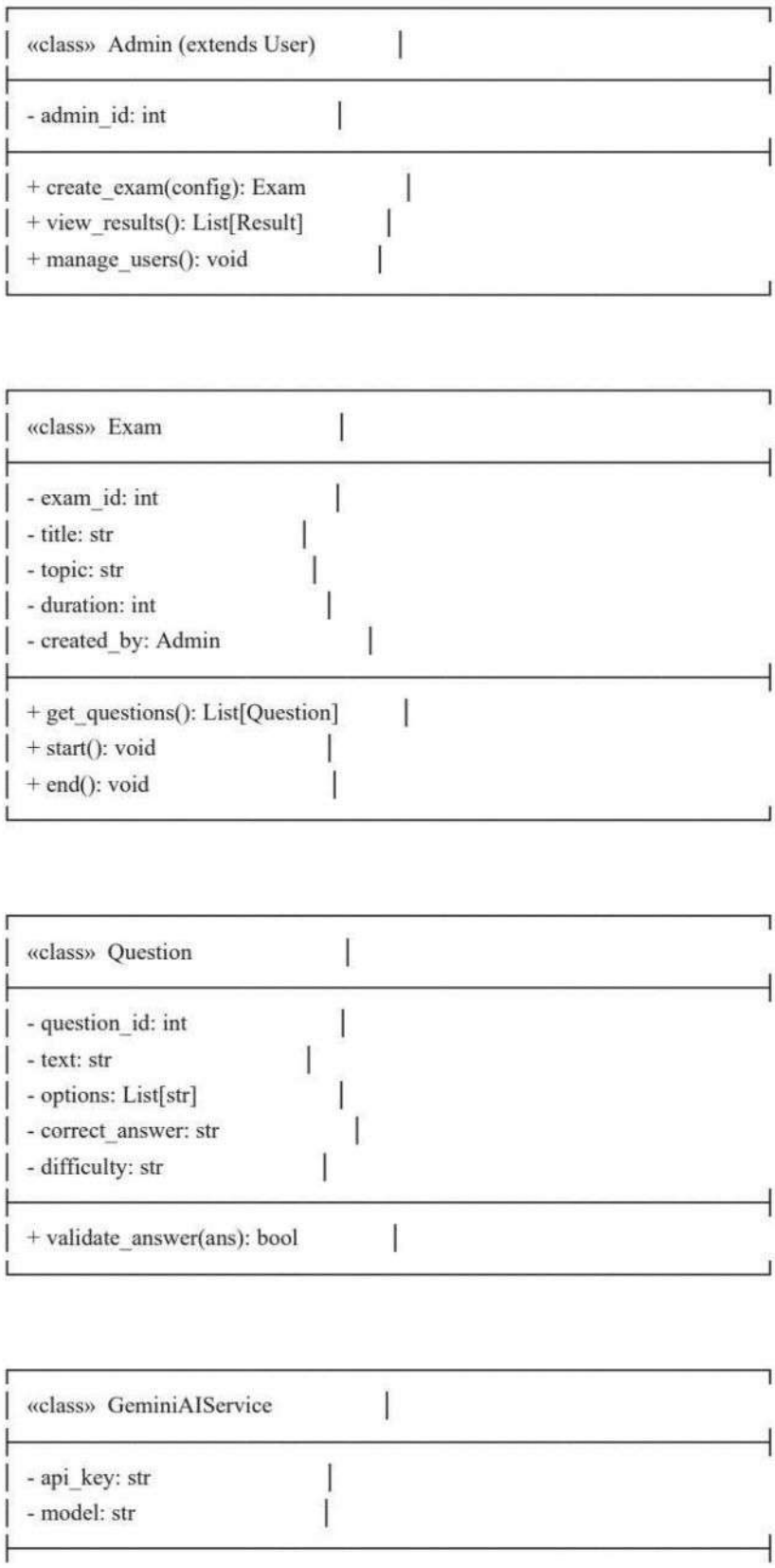
Use Case	Actor(s)	Description
Login / Authenticate	Student, Admin	User logs in using credentials; voice prompt guides visually impaired users
Receive Voice Instructions	Student	System reads aloud exam rules and navigation tips
Navigate Exam (Keyboard-only)	Student	All UI elements reachable by Tab/Arrow/Enter keys

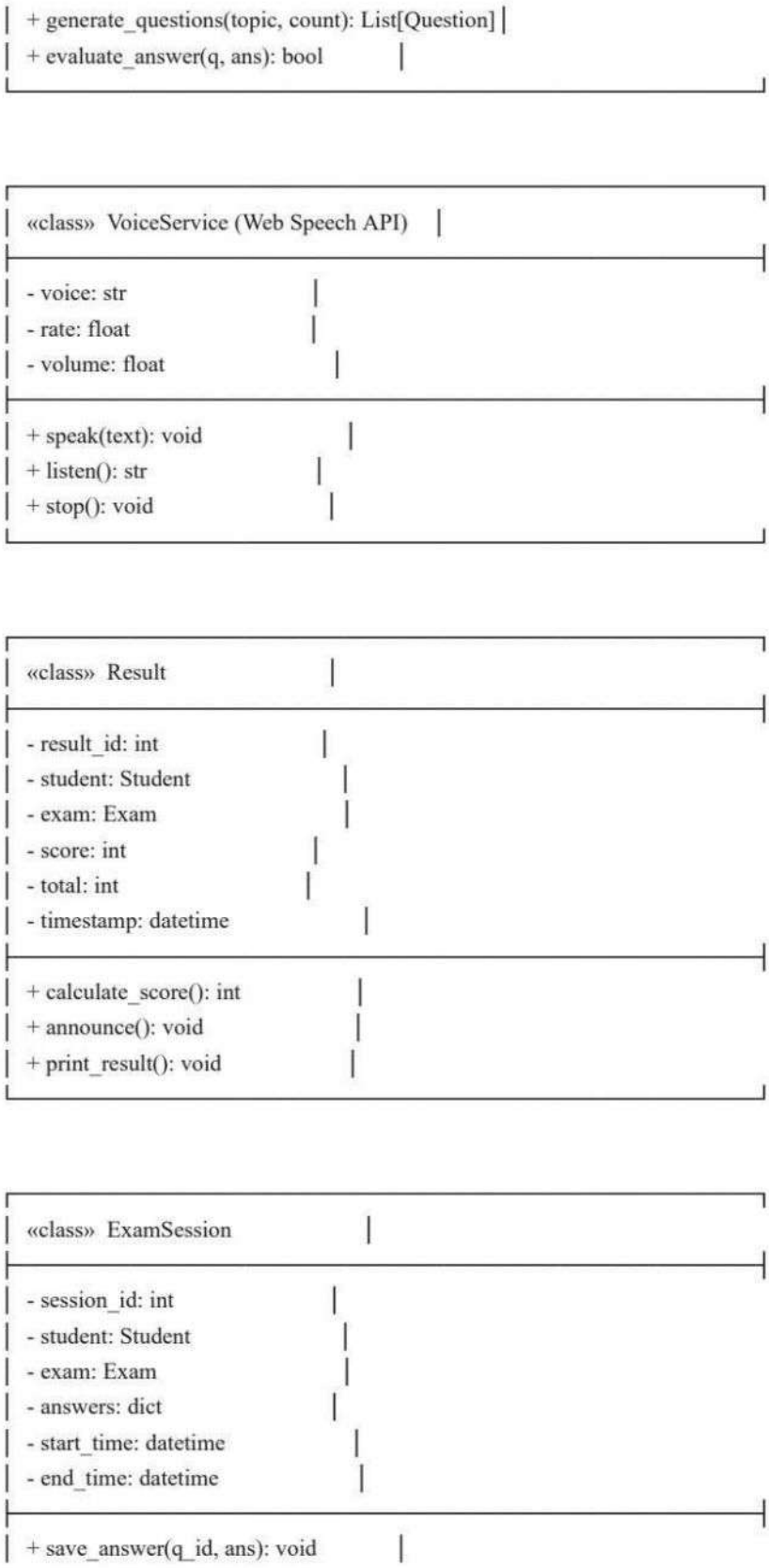
Answer AI-Generated Questions	Student, Gemini AI	Questions are unique each session, generated via Gemini API
Submit Exam	Student	Student submits; system auto-evaluates answers
Announce Result via Voice	Student, Web Speech API	Result and score read aloud automatically
Print Result	Student	Printable result page generated
Manage Users	Admin	Admin creates/modifies/deletes student accounts
Configure Exam	Admin	Set topics, number of questions, time limit
View & Export Results	Admin	Admin views all student results; export as report

6.2. Class Diagram

The Class Diagram shows the static structure of the EASVI application — its classes, attributes, methods, and relationships.




«class» Admin (extends User)
- admin_id: int
+ create_exam(config): Exam
+ view_results(): List[Result]
+ manage_users(): void
«class» Exam
- exam_id: int
- title: str
- topic: str
- duration: int
- created_by: Admin
+ get_questions(): List[Question]
+ start(): void
+ end(): void
«class» Question
- question_id: int
- text: str
- options: List[str]
- correct_answer: str
- difficulty: str
+ validate_answer(ans): bool
«class» GeminiAIService
- api_key: str
- model: str


+ generate_questions(topic, count): List[Question]
+ evaluate_answer(q, ans): bool
«class» VoiceService (Web Speech API)
- voice: str
- rate: float
- volume: float
+ speak(text): void
+ listen(): str
+ stop(): void
«class» Result
- result_id: int
- student: Student
- exam: Exam
- score: int
- total: int
- timestamp: datetime
+ calculate_score(): int
+ announce(): void
+ print_result(): void
«class» ExamSession
- session_id: int
- student: Student
- exam: Exam
- answers: dict
- start_time: datetime
- end_time: datetime
+ save_answer(q_id, ans): void

```
| + submit(): Result                    |
| + get_elapsed_time(): int             |
└──────────────────────────────────────────┘
```

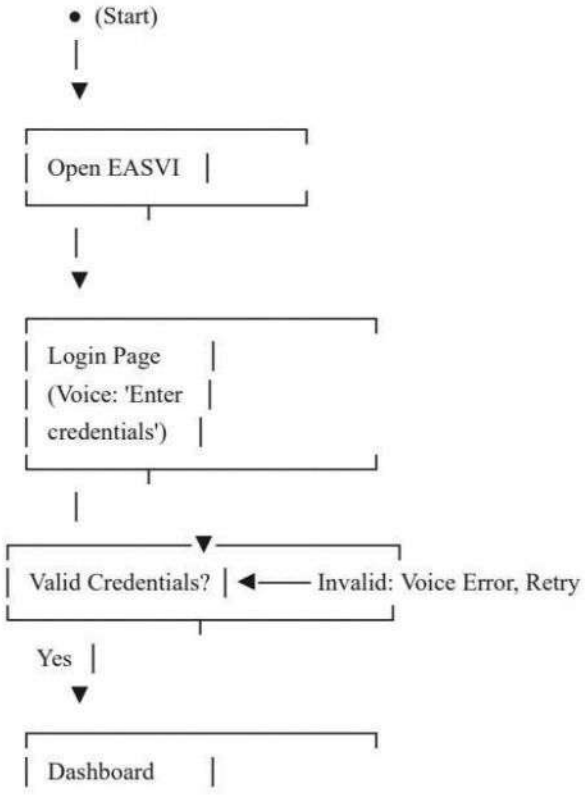
Relationships

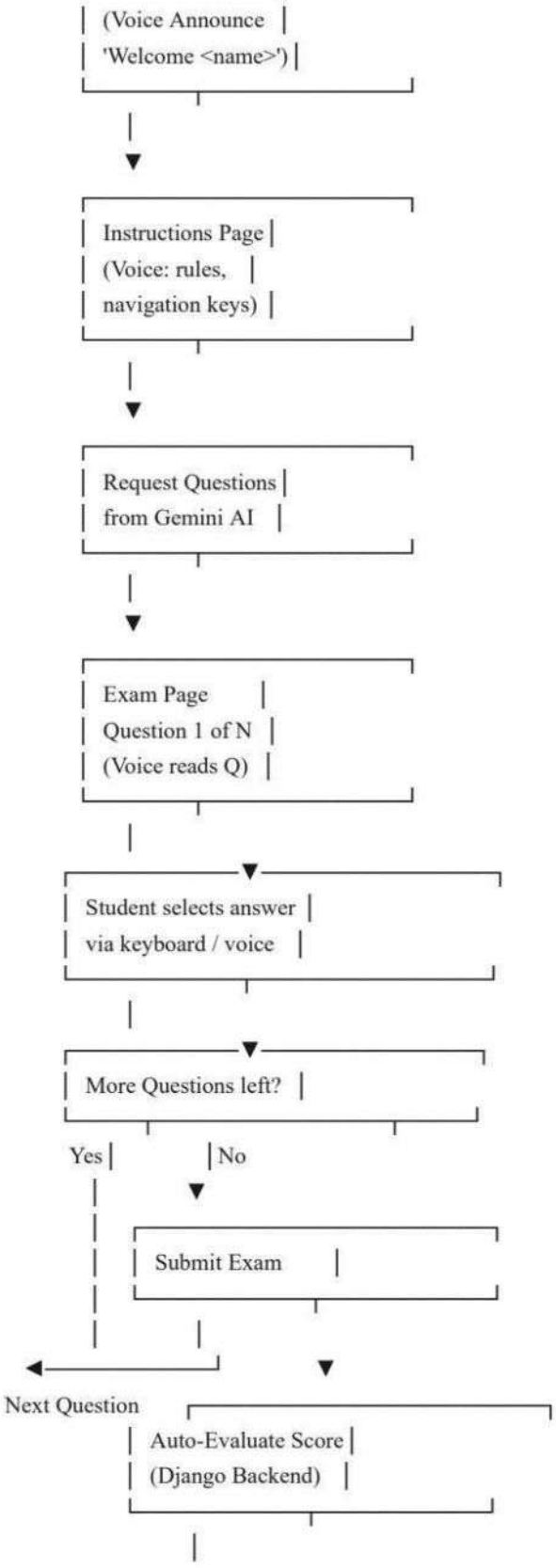
From	Relationship	To	Multiplicity
Student	extends	User	—
Admin	extends	User	—
Admin	creates	Exam	1..*
Exam	contains	Question	1..*
ExamSession	belongs to	Student	1
ExamSession	uses	Exam	1
ExamSession	generates	Result	1
GeminiAIService	provides	Question	0..*
VoiceService	used by	ExamSession	1
VoiceService	used by	Result	1

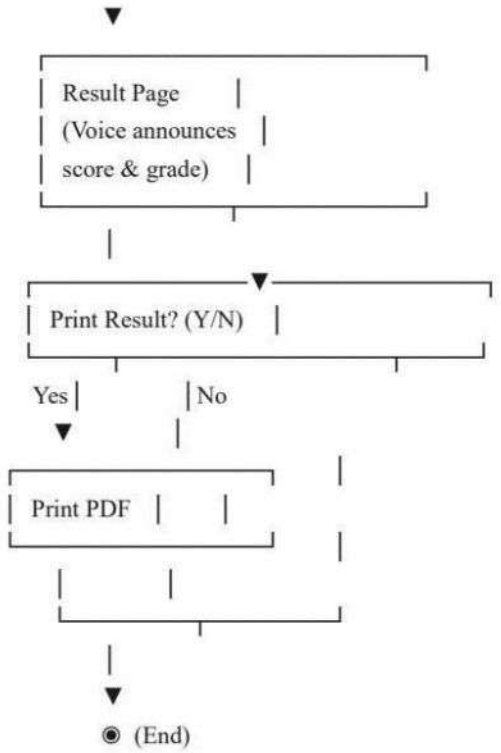
6.3. Activity Diagram

The Activity Diagram describes the workflow / business logic of the main examination process in EASVI.

A. Main Examination Workflow

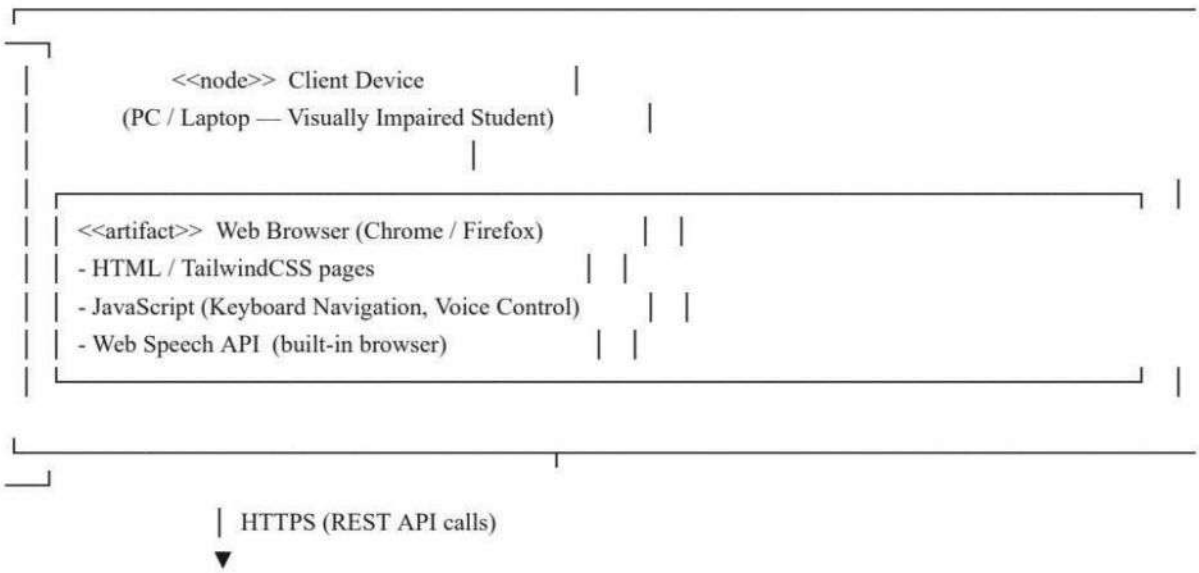


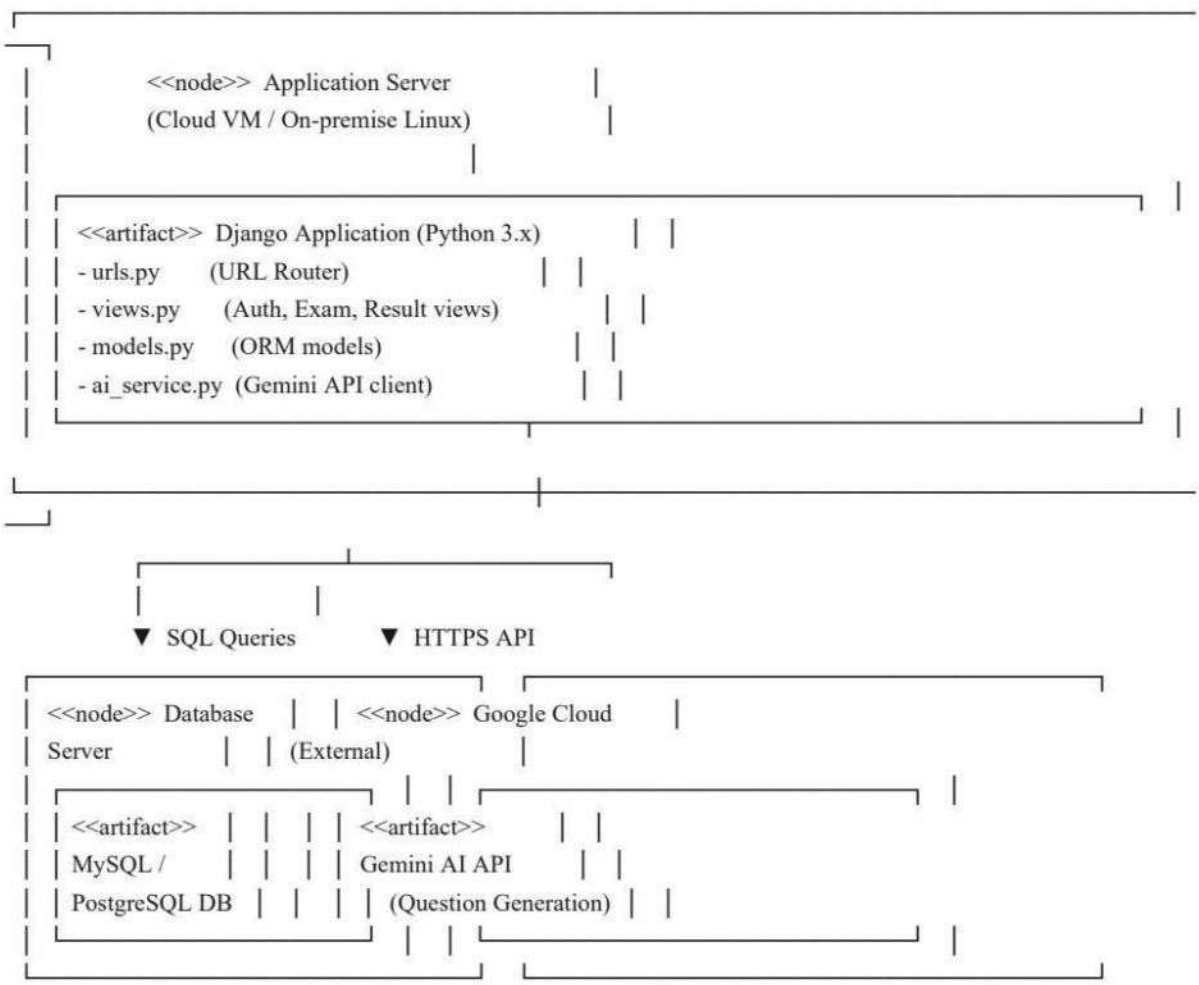

(Voice Announce
'Welcome <name>')
Instructions Page
(Voice: rules,
navigation keys)
Request Questions
from Gemini AI
Exam Page
Question 1 of N
(Voice reads Q)
Student selects answer
via keyboard / voice
More Questions left?
Yes
No
Submit Exam
Next Question
Auto-Evaluate Score
(Django Backend)



6.4. Deployment Diagram

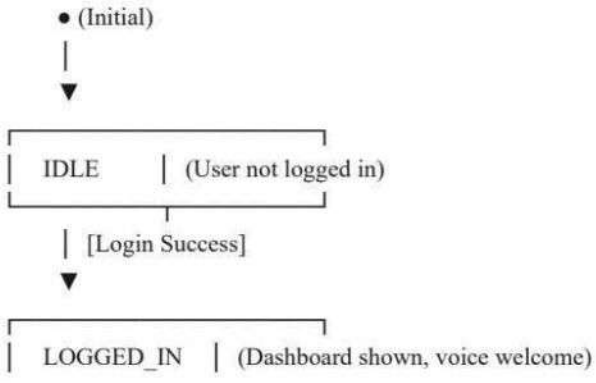
The Deployment Diagram shows how the physical/virtual nodes are arranged and which artefacts run where.

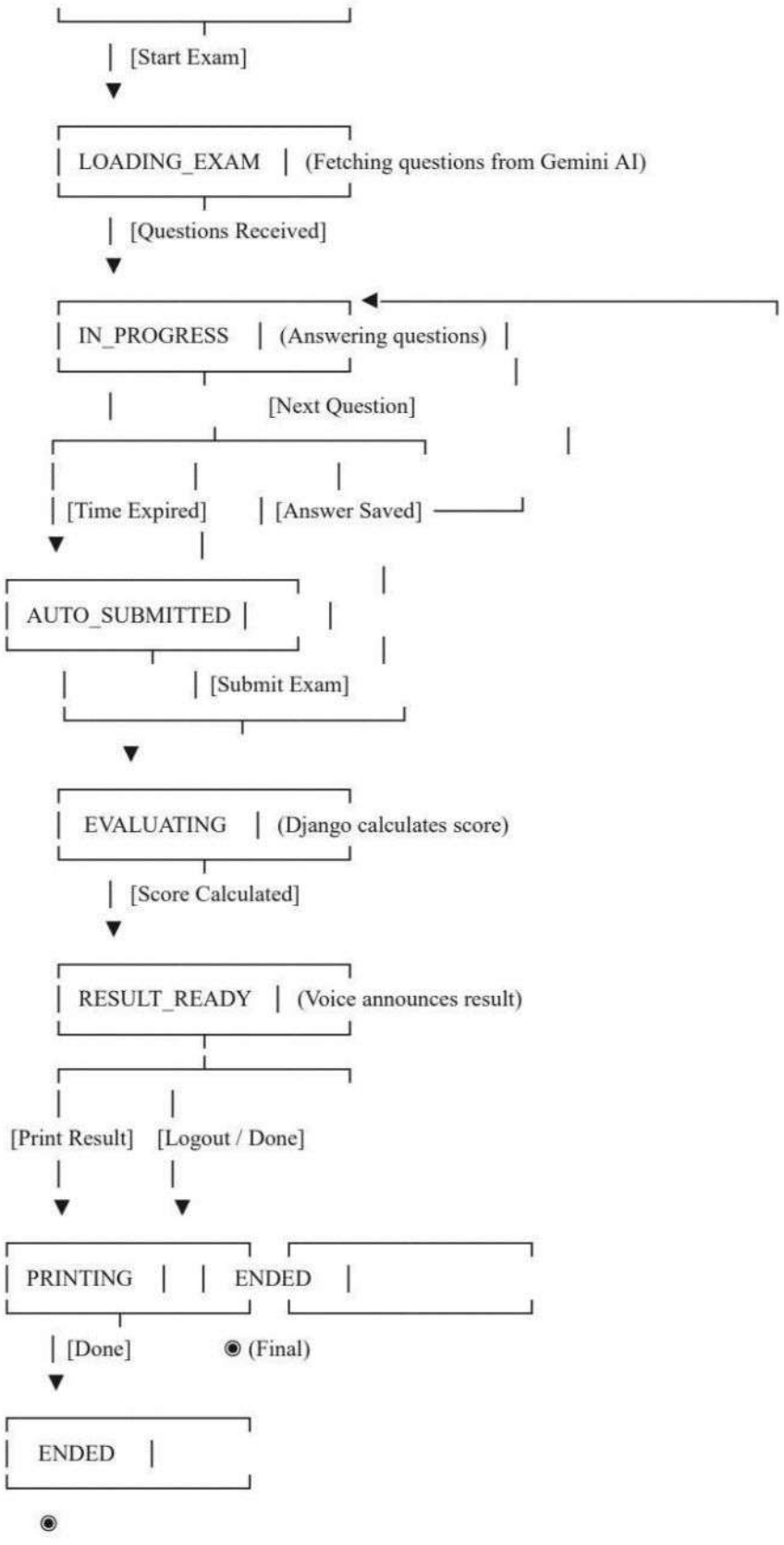




6.5. State Diagram

The State Diagram models the states of an ExamSession object through its lifecycle.




[Start Exam]
LOADING_EXAM
(Fetching questions from Gemini AI)
[Questions Received]
IN_PROGRESS
(Answering questions)
[Next Question]
[Time Expired]
[Answer Saved]
AUTO_SUBMITTED
[Submit Exam]
EVALUATING
(Django calculates score)
[Score Calculated]
RESULT_READY
(Voice announces result)
[Print Result]
[Logout / Done]
PRINTING
ENDED
[Done]
(Final)
ENDED

State Descriptions

State	Description	Transitions Out
IDLE	No active session	Login Success → LOGGED_IN
LOGGED_IN	User authenticated; dashboard visible	Start Exam → LOADING_EXAM
LOADING_EXAM	Waiting for Gemini to generate questions	Questions Received → IN_PROGRESS
IN_PROGRESS	Student answering questions one by one	Submit → EVALUATING; Time Expired → AUTO_SUBMITTED
AUTO_SUBMITTED	Timer ran out; system auto-submits	→ EVALUATING
EVALUATING	Backend scores answers	Score Ready → RESULT_READY
RESULT_READY	Score available; voice announcement triggered	Print → PRINTING; Logout → ENDED
PRINTING	PDF result being generated	Done → ENDED
ENDED	Session complete	(Final State)

7. DATA REQUIREMENTS

7.1 Data Entities Description

The following entities constitute the core data model of the EASVI system. All entities are managed through the Django ORM and persisted in the relational database.

7.1.1 User

- Represents both students and administrators. Role-based differentiation via the 'role' enum field.
- Sensitive fields: password_hash (bcrypt), email — never returned in API responses.

7.1.2 Exam

- Represents a configured examination template created by an administrator.
- Linked to the Admin who created it via a foreign key.

7.1.3 Question

- Stores AI-generated questions associated with an exam session. Not permanently stored between sessions unless caching is enabled.
- Each question includes text, four options, and the correct answer identifier.

7.1.4 ExamSession

- Represents a single student's attempt at an exam. Records all answers, timestamps, and session state.
- Linked to both a Student and an Exam via foreign keys.

7.1.5 Result

- Generated upon exam submission. Stores the final score and is linked to the ExamSession.
- Immutable after creation; no modifications permitted post-evaluation.

7.2 Database Schema Design

Table	Column	Data Type	Constraints / Notes
users	user_id	INT	PRIMARY KEY, AUTO_INCREMENT
users	username	VARCHAR(50)	UNIQUE, NOT NULL
users	password_hash	VARCHAR(255)	NOT NULL (bcrypt)
users	email	VARCHAR(100)	UNIQUE, NOT NULL
users	role	ENUM('student','admin')	NOT NULL, DEFAULT 'student'
users	is_active	BOOLEAN	DEFAULT TRUE
users	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
exams	exam_id	INT	PRIMARY KEY, AUTO_INCREMENT
exams	title	VARCHAR(200)	NOT NULL
exams	topic	VARCHAR(100)	NOT NULL
exams	duration_minutes	INT	NOT NULL, DEFAULT 30
exams	question_count	INT	NOT NULL, DEFAULT 10
exams	difficulty	ENUM('easy','medium','hard')	NOT NULL, DEFAULT 'medium'
exams	created_by	INT	FOREIGN KEY → users.user_id

exam_sessions	session_id	INT	PRIMARY KEY, AUTO_INCREMENT
exam_sessions	student_id	INT	FOREIGN KEY → users.user_id
exam_sessions	exam_id	INT	FOREIGN KEY → exams.exam_id
exam_sessions	answers	JSON	Stores {question_id: selected_option}
exam_sessions	start_time	DATETIME	NOT NULL
exam_sessions	end_time	DATETIME	NULLABLE (set on submission)
exam_sessions	status	ENUM('in_progress','submitted','evaluated')	NOT NULL
results	result_id	INT	PRIMARY KEY, AUTO_INCREMENT
results	session_id	INT	FOREIGN KEY → exam_sessions.session_id, UNIQUE
results	score	INT	NOT NULL
results	total	INT	NOT NULL
results	percentage	DECIMAL(5, 2)	NOT NULL
results	timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP

7.3 Data Dictionary

Field Name	Type	Description and Business Rules
user_id	INT	System-generated unique identifier for each user. Never reused.
username	VARCHAR(50)	User-chosen login identifier. Alphanumeric and underscore only. Minimum 4 characters.
password_hash	VARCHAR(255)	bcrypt hash of the user's password. Minimum 8-character password enforced at application layer.
role	ENUM	'student' or 'admin'. Determines accessible features and dashboard view.
is_active	BOOLEAN	Soft-delete flag. Inactive users cannot log in but their data is retained.

topic	VARCHAR(100)	Subject area for AI question generation (e.g., 'Python Programming', 'Mathematics').
duration_minutes	INT	Maximum time allowed for the exam in minutes. Range: 5 to 180.
difficulty	ENUM	Passed to Gemini API prompt. 'easy', 'medium', or 'hard'. Affects question complexity.
answers	JSON	Key-value map of {question_id: option_letter} representing student's selections during the session.
status	ENUM	Tracks exam session state machine transitions: in_progress → submitted → evaluated.
score	INT	Number of correctly answered questions. Range: 0 to total.
percentage	DECIMAL(5,2)	Calculated as (score / total) * 100. Stored to 2 decimal places.

8. APPENDICES

8.1 Key UI Pages Description

Page	Description and Accessibility Notes
Login Page	Minimal two-field form (username, password) with clear labels. Voice announces instructions on load. Login button is keyboard-accessible. Error messages announced via ARIA live regions.
Registration Page	Five-field form for new user signup. Each field announced in sequence. Real-time validation with voice feedback. Password strength indicator described verbally.
Dashboard	Post-login landing page. Voice announces 'Welcome, [Name]. Press Tab to navigate to Start Exam, or press H for help.' Single primary CTA button for starting exam.
Instructions Page	Full-screen voice reading of exam rules and keyboard shortcuts. User must confirm readiness before exam begins. Skip option available for experienced users.
Exam Page	Single question displayed at a time. Question number, text, and all four options read aloud. Answer selection confirmed by voice. Timer displayed (for sighted users) and announced at 5-minute and 1-minute warnings.

Result Page	Displays score, percentage, grade, and question-by-question breakdown. Full result announced via voice on page load. Re-read and print options keyboard accessible.
Admin Dashboard	Tabular view of all students, exams, and results. Exam configuration form. CSV/PDF export buttons. Standard web UI; no special accessibility requirements.

8.2 Future Enhancements

The following enhancements are identified as valuable extensions beyond the current scope and are recommended for future versions of EASVI:

- Multi-Language Support: Voice instructions and TTS output in regional languages (e.g., Hindi, Tamil, Bengali) to serve a broader student population.
- Mobile Application: Native Android and iOS applications providing the same accessible examination experience with enhanced mobile screen reader integration.
- Braille Display Integration: Support for refreshable Braille displays as an output channel for students who prefer tactile reading.
- Adaptive Difficulty: Real-time adjustment of question difficulty based on student performance during the exam session.
- LMS Integration: REST API connectors for integration with Moodle, Canvas, or other Learning Management Systems used by educational institutions.
- Offline Mode: Ability to cache questions locally and submit when connectivity is restored, for use in low-bandwidth environments.
- Detailed Analytics Dashboard: Performance trend analysis for administrators, including topic-wise accuracy, average time per question, and cohort comparison.
- Multi-Format Questions: Support for question types beyond multiple-choice, including true/false, short answer (voice-recorded), and numerical input.
- Proctoring Features: Basic anti-cheating measures such as session monitoring and tab-switch detection, with accessible implementation.

8.3 Known Limitations

The following limitations are acknowledged in the current version of EASVI:

- Browser Dependency: The Web Speech API is not uniformly implemented across all browsers. Chrome provides the most complete implementation. Firefox has limited speech recognition support. Safari has inconsistent behavior.
- Internet Dependency: The system requires an active internet connection at all times. There is no offline fallback in the current version.
- Gemini API Dependency: Question quality, formatting, and relevance are dependent on the Gemini model's output. Edge cases may produce malformed questions requiring retry logic.

- Voice Language Support: The default voice for TTS is system-dependent and may vary across operating systems. No custom voice selection is available in the current version.
- Single Language: The current version supports English only for both the UI and AI-generated questions.
- No Real-Time Proctoring: The system does not include webcam-based or AI-based proctoring. It relies on institutional honor code compliance.
- Admin Dashboard Accessibility: The administrator dashboard is not designed to be screen-reader-accessible, as administrators are assumed to have unimpaired vision. This should be addressed in a future release.

8.4 Revision History

Version	Date	Author(s)	Changes
1.0	2026	Mayank Mitra, Mishika Kasliwal	Initial release — full SRS document

SOFTWARE DEVELOPMENT LIFECYCLE

Examination System for Visually Impaired (EASVI)



By :
Mayank Mitra (EN23CS301604)
Mishika Kasliwal (EN23CS301617)

Section : 6K
Session : Jan–Jun 2026

Under the Guidance of
Prof. Garima Tukra
Dr . Sheetal Bawane

1. Introduction

This document defines the Software Development Lifecycle (SDL) for the Examination System for Visually Impaired (EASVI). It outlines all phases of development — from requirement gathering through deployment and maintenance — providing a structured plan for building a web-based, AI-powered, accessible examination platform.

1.1 Project Overview

Attribute	Details
Project Name	EASVI — Examination System for Visually Impaired
Type	Web-based, AI-powered accessible examination platform
Tech Stack	Django (Python), HTML5/TailwindCSS, JavaScript, Gemini AI API, Web Speech API
Database	MySQL 8.0+ / PostgreSQL 14+
Standard	IEEE 830-1998 SRS Compliant
Development Model	Agile — Iterative & Incremental (4 sprints)
Target Users	Visually Impaired Students, Administrators

1.2 SDL Model Selection

EASVI adopts the Agile Iterative Model. This choice is justified by:

- Accessibility requirements that need continuous user feedback and refinement
- Integration with external APIs (Gemini, Web Speech) that require iterative testing
- Evolving accessibility standards (WCAG 2.1) requiring flexible adjustments
- Phased delivery enabling early usable builds for stakeholder review

2. SDL Phases Overview

The EASVI SDL is structured into 7 sequential phases. Each phase has defined inputs, activities, outputs, and entry/exit criteria to ensure quality and traceability throughout the lifecycle.

Phase	Activity	Output/Deliverable	Duration
Phase 1	Requirements Analysis	SRS Document (IEEE 830)	Week 1–2

Phase	Activity	Output/Deliverable	Duration
Phase 2	System Design	Architecture, DB Schema, UML Diagrams	Week 3–4
Phase 3	Sprint 1 — Auth & Voice	Login/Signup + Voice System	Week 5–6
Phase 4	Sprint 2 — Exam Engine	AI Question Gen + Navigation	Week 7–8
Phase 5	Sprint 3 — Evaluation	Auto-eval + Result Announcement	Week 9–10
Phase 6	Sprint 4 — Admin & QA	Admin Dashboard + Full Testing	Week 11–12
Phase 7	Deployment & Maintenance	Live System + Maintenance Plan	Week 13–14

3. Detailed Phase Descriptions

Phase 1 — Requirements Analysis

3.1 Objectives

- Identify all stakeholder needs: visually impaired students, admins, faculty
- Document functional requirements (F1–F10) and non-functional requirements (NFR-P01–P06)
- Define system scope, constraints, assumptions, and external dependencies

3.2 Key Activities & Outputs

Activity	Output
Stakeholder interviews & accessibility research	Stakeholder needs register
Define functional requirements (auth, voice, exam, AI, eval)	IEEE 830-1998 SRS document
Define non-functional requirements (performance, security)	NFR specification table
Risk identification (API dependencies, browser support)	Risk register

3.3 Core Functional Requirements Captured

Req. ID	Description
F1	Secure user authentication with voice-guided login/signup
F2–F3	Contextual voice instruction system and exam initialization
F4	AI-based dynamic question generation via Gemini API
F5–F6	Keyboard-only navigation and multi-modal answer recording
F7–F8	Automatic evaluation and voice-announced result delivery
F9–F10	Printable result output and admin management dashboard

Phase 2 — System Design

3.4 Architecture Design

EASVI follows a 3-tier architecture. The presentation layer renders accessible HTML/CSS/JS in the browser. The application layer runs Django business logic. The data layer uses MySQL/PostgreSQL via Django ORM.

Layer	Technology & Responsibility
Presentation	HTML5, TailwindCSS, JavaScript — Accessible UI, keyboard nav, Web Speech API
Application	Django 4.x (Python 3.9+) — Auth, exam logic, AI integration, evaluation
Data	MySQL 8.0 / PostgreSQL 14 — Users, exams, sessions, results
AI Integration	Google Gemini API — Dynamic question generation (JSON format)
Accessibility	Web Speech API (browser-native) — TTS output, STT input

3.5 Database Schema Summary

Table	Key Columns & Constraints
users	user_id (PK), username (UNIQUE), password_hash (bcrypt), role ENUM(student/admin)
exams	exam_id (PK), topic, duration_minutes, question_count, difficulty ENUM, created_by (FK)
exam_sessions	session_id (PK), student_id (FK), exam_id (FK), answers (JSON), status ENUM
results	result_id (PK), session_id (FK, UNIQUE), score, total, percentage DECIMAL(5,2)

3.6 UML Models Designed

- Use Case Diagram — Student and Admin interactions with system and external APIs
- Class Diagram — User, Student, Admin, Exam, Question, ExamSession, Result, VoiceService, GeminiAIService
- Activity Diagram — Full exam workflow: login → questions → submit → evaluate → result
- State Diagram — ExamSession states: IDLE → LOGGED_IN → LOADING → IN_PROGRESS → EVALUATING → RESULT_READY → ENDED
- Deployment Diagram — Client browser, Django app server, DB server, Google Cloud (Gemini API)

Sprint 1 — Authentication & Voice System (Phase 3)

3.7 Sprint Goals

- Implement secure user authentication (UC-01 Login, UC-02 Registration)
- Implement full Voice Interaction System (FR-V01 through FR-V07)
- Establish keyboard-only navigation across all accessible UI pages

3.8 Implementation Details

Component	Technical Specification
Login Module	Django session auth with bcrypt; account locked after 5 failed attempts
Registration Module	Username uniqueness check, email validation, password strength (min 8 chars)
TTS Engine	Web Speech API SpeechSynthesis — all page content, errors, confirmations read aloud
STT Engine	Web Speech API SpeechRecognition — optional voice answer input
Voice Queue	Sequential speech queue; no overlapping; 300ms trigger threshold (FR-V07)
Keyboard Nav	Tab/Enter/Arrow/R(re-read)/Space(pause) shortcuts; ARIA labels on all elements

3.9 Acceptance Criteria

- Login page announces instructions within 300ms of page load
- Failed login triggers voice error and clears password field
- User can complete full registration using keyboard alone

- Voice speed and volume adjustable via keyboard-accessible settings

Sprint 2 — Exam Engine & AI Question Generation (Phase 4)

3.10 Sprint Goals

- Implement exam initialization flow (UC-03) with Gemini API integration
- Build AI question generation service (FR-Q01 through FR-Q07)
- Implement full answer submission and navigation module (UC-04)

3.11 Gemini API Integration

Requirement	Implementation
FR-Q01/Q02	Prompt specifies topic, difficulty, count; expects JSON with question, options A-D, correct answer
FR-Q03	Response validation: checks format, 4 options, correct answer identifier present
FR-Q04	Session-level deduplication using question_id set per ExamSession
FR-Q05	Structured prompt: { topic, difficulty: easy/medium/hard, count, format: JSON }
FR-Q06	Questions cached in session; no re-fetch during navigation
FR-Q07	API key stored as server-side environment variable; never sent to client

3.12 Exam Navigation

- Keys: A/B/C/D or arrows to select; Enter to confirm; N/P for next/previous question
- Voice confirms selection: 'You selected Option X: [text]. Press Enter to confirm.'
- Timer displayed visually; voice warnings at 5-minute and 1-minute remaining
- Auto-submission on timer expiry; all recorded answers preserved

Sprint 3 — Evaluation & Result System (Phase 5)

3.13 Automatic Evaluation Engine

Evaluation is performed entirely server-side in Django to prevent client manipulation. Requirements FR-E01 through FR-E06 govern this module.

Req. ID	Evaluation Rule
FR-E01	Compare each submitted answer against correct_answer from session data
FR-E02	Calculate raw score (correct count) and percentage = (score/total)*100
FR-E03	Evaluation and result persistence must complete within 2 seconds of submission
FR-E04	Binary scoring only — correct or incorrect; no partial credit in v1.0
FR-E05	Stateless evaluation engine — identical inputs always produce identical outputs
FR-E06	Result immediately persisted to results table; immutable post-creation

3.14 Result Delivery

- Voice announces: score, percentage, grade, and question-by-question summary
- Result page provides re-read option (keyboard: R) and print option
- Print generates formatted result document with student details and score breakdown

Sprint 4 — Admin Dashboard & Quality Assurance (Phase 6)

3.15 Admin Features

- User management: create, modify, deactivate student accounts (is_active flag)
- Exam configuration: topic, question count, difficulty, duration per exam
- Result viewing and export: tabular view of all sessions; CSV/PDF export

3.16 Testing Strategy

Test Type	Scope & Tools
Unit Testing	Django TestCase for all views, models, evaluation engine; pytest
Integration Testing	Gemini API mock responses; Web Speech API browser simulation
Accessibility Testing	NVDA + Chrome, JAWS, VoiceOver; keyboard-only navigation audit
Performance Testing	100 concurrent sessions (NFR-P05); page load < 3s (NFR-P01)

Test Type	Scope & Tools
Security Testing	OWASP checklist: bcrypt passwords, session tokens, API key protection, CSRF
UAT	Test sessions with visually impaired users; feedback-driven refinements

3.17 Non-Functional Requirements Verification

NFR ID	Requirement & Verification Method
NFR-P01	Page load < 3s on 10 Mbps — load testing with Locust
NFR-P02	Voice feedback within 300ms — automated timing assertions
NFR-P03	Gemini API response < 5s — API timeout monitoring
NFR-P04	Evaluation < 2s — Django test timing assertions
NFR-P05	100 concurrent sessions — Locust/JMeter load simulation
NFR-P06	DB queries < 500ms — Django Debug Toolbar profiling

Phase 7 — Deployment & Maintenance

3.18 Deployment Architecture

Component	Specification
Application Server	Ubuntu 20.04 LTS, Python 3.9+, Django 4.x, Gunicorn/Nginx
Database Server	MySQL 8.0+ or PostgreSQL 14+; min 4GB RAM, 20GB storage
Network	HTTPS (TLS 1.2+); outbound access to Gemini API endpoints
Client	Chrome 90+, Firefox 88+, Edge 90+ with Web Speech API support
Environment Config	Gemini API key, DB credentials via .env; never in source code

3.19 Deployment Checklist

- Run django migrate to initialize all database tables
- Create superuser account for initial admin access
- Configure .env file with GEMINI_API_KEY and database credentials
- Collect static files and serve via Nginx
- Verify Web Speech API availability in target browser environments
- Run full regression test suite in production environment

3.20 Maintenance Plan

Maintenance Type	Activities
Corrective	Bug fixes from UAT feedback; voice announcement edge cases; API error handling
Adaptive	Updates for Gemini API version changes; browser Web Speech API changes
Perfective	Multi-language TTS (Hindi, Tamil), mobile app, offline mode (per Section 8.2)
Preventive	Regular security patches; database backups; performance monitoring

4. Risk Management

Risk	Mitigation Strategy
Gemini API quota / deprecation	Retry logic (3 attempts); session abort with voice error; monitor Google API changelog
Web Speech API browser inconsistency	Detect API on load; graceful degradation to text-only fallback; Chrome recommended
Performance under 100+ concurrent users	Connection pooling; Django ORM query optimization; caching layer
Accessibility gaps for diverse impairments	Test with multiple screen readers (NVDA, JAWS, VoiceOver); WCAG 2.1 audit
AI-generated malformed questions	JSON validation layer; re-request up to 3 times; log malformed responses
Security — API key exposure	Server-side env variable; never in client JS; HTTPS only

5. Development Timeline Summary

Week	Milestone
Week 1–2	Requirements finalized — IEEE 830 SRS document complete
Week 3–4	System design complete — architecture, DB schema, all UML diagrams
Week 5–6	Sprint 1 complete — Auth module + Voice system functional
Week 7–8	Sprint 2 complete — Gemini AI integration + Exam navigation live
Week 9–10	Sprint 3 complete — Evaluation engine + Result announcement working
Week 11–12	Sprint 4 complete — Admin dashboard + Full test suite passed
Week 13–14	Deployment live — UAT complete, documentation finalized

6. Known Limitations & Future Enhancements

6.1 Known Limitations (v1.0)

- Browser dependency: Web Speech API most reliable on Chrome; Firefox/Safari partial support

- Internet dependency: No offline fallback in current version
- English-only: UI and AI-generated questions in English only
- Admin dashboard not screen-reader accessible (assumed sighted admin users)
- No real-time proctoring or webcam-based monitoring

6.2 Planned Future Enhancements

- Multi-language TTS support (Hindi, Tamil, Bengali) for broader accessibility
- Native Android/iOS mobile application with screen reader integration
- Offline mode with local question caching and deferred submission
- Adaptive difficulty: real-time adjustment based on student performance
- LMS integration (Moodle, Canvas) via REST API connectors
- Detailed analytics dashboard for administrators

7. Document Revision History

Version	Details
v1.0 — 2026	Initial SDL document — Mayank Mitra & Mishika Kasliwal, Medicaps University

EASVI: An AI-Powered Web-Based Examination System for Visually Impaired Students

Mayank Mitra, Mishika Kasliwal

Department of Computer Science & Engineering, Medi-Caps University, Indore, India

Under the Guidance of Prof. Garima Tukra & Dr. Sheetal Bawane

Abstract

The Examination System for Visually Impaired (EASVI) is an AI-powered, web-based platform engineered to deliver an inclusive and fully accessible examination experience for students with visual disabilities. Conventional online examination systems depend heavily on visual interfaces and mouse-based navigation, creating fundamental accessibility barriers for blind and visually impaired users. EASVI systematically addresses these deficiencies through the integration of voice-based interaction via the Web Speech API, complete keyboard-only navigation, and AI-driven question generation using Google's Gemini language model. The system is architected using HTML5, TailwindCSS, and JavaScript on the front-end, with Django serving as the backend web framework and MySQL/PostgreSQL for data persistence. Dynamic question generation via the Gemini API ensures each examination session presents a unique question set, promoting academic integrity. Results are evaluated instantly upon submission and announced via synthesized speech, eliminating manual delays. Comparative analysis against existing accessible examination tools—including JAWS screen reader, ATutor, and ProctorTrack—demonstrates that EASVI offers a comprehensive, purpose-built solution that surpasses current alternatives in terms of voice integration, automation, and AI capability. The system aligns with the Web Content Accessibility Guidelines (WCAG 2.1) and the United Nations Sustainable Development Goal 4 (Quality Education), and represents a meaningful contribution to the field of inclusive educational technology.

Keywords: *Visually Impaired, Accessibility, Web Speech API, Gemini AI, Inclusive Examination, Voice Interaction, Django, Keyboard Navigation, Text-to-Speech, WCAG*

1. Introduction

Examinations constitute a fundamental pillar of academic evaluation, serving as the primary mechanism through which institutions assess the knowledge, skills, and academic progress of students. The proliferation of digital technologies has accelerated the transition from paper-based to online examination formats; however, this transition has largely neglected the accessibility requirements of students with visual impairments. Despite significant advances in web development and assistive technology, most contemporary online examination platforms are designed with a sighted user as the default, requiring mouse interaction, visual reading of question text, and interpretation of graphical interface elements—tasks that are inherently inaccessible to individuals who are blind or have severe visual impairment [1].

This accessibility gap translates directly into academic inequity. Visually impaired students are routinely compelled to rely on human scribes to navigate examinations, a process that introduces logistical dependencies, privacy concerns, and inconsistencies in examination conditions. Furthermore, the absence of voice-guided interfaces and assistive technologies tailored to online assessments limits the autonomy and academic participation of this demographic, contributing to broader patterns of social exclusion in educational settings.

The Examination System for Visually Impaired (EASVI) is developed to comprehensively address this accessibility deficit. EASVI is a fully web-based, AI-integrated examination platform that enables visually impaired students to independently participate in online examinations without any reliance on scribes or external human assistance. The system employs voice-based interaction through the Web Speech API to synthesize and deliver all examination content—questions, instructions, navigation cues, and results—in audible speech. Navigation across the entire system is accomplished through keyboard shortcuts, eliminating any requirement for mouse-based input at any stage of the examination process.

A distinctive feature of EASVI is its integration with Google's Gemini large language model for the automatic generation of examination questions. This mechanism ensures that each examination session presents a unique and dynamically generated set of questions, significantly enhancing academic integrity and reducing the risk of question leakage. The system's backend, built on the Django web framework, manages user authentication, session control, question delivery, and result computation, while a relational database (MySQL or PostgreSQL) provides persistent storage for user profiles, examination metadata, and results.

2. Literature Review

A substantial body of research and a number of practical tools have been developed to address accessibility challenges in educational technology for visually impaired individuals. A systematic review of the existing landscape reveals both the progress achieved and the critical gaps that persist.

2.1 Screen Reader Technologies

JAWS (Job Access With Speech), developed by Freedom Scientific, is the most widely deployed screen reading software globally. It converts on-screen text into synthesized speech and provides keyboard-based navigation for general computer interaction [7]. While JAWS enables visually impaired users to interact with computers broadly, it is not purpose-built for online examination environments and requires extensive configuration to function with examination platforms, many of which are not designed to be JAWS-compatible. Research by Yesilada et al. [5] has documented significant limitations in screen reader compatibility with standard web content, particularly complex interactive elements such as timed assessments.

2.2 Accessible Learning Management Systems

ATutor is an open-source Learning Management System (LMS) explicitly designed with accessibility as a priority [8]. It provides keyboard navigation support and basic screen reader compatibility, making it more accessible than most mainstream LMS platforms. However, ATutor is fundamentally a learning delivery platform rather than an examination system. It lacks examination-specific features such as timed assessments, dynamic question generation, instant result computation, and voice-guided question delivery.

2.3 Remote Proctoring Solutions

ProctorTrack, developed by Verificient Technologies, is a remote proctoring solution that includes limited support for screen readers and keyboard navigation [9]. However, its primary function is examination invigilation rather than accessibility facilitation. It does not provide voice-guided question delivery, AI-generated questions, or an end-to-end accessible examination experience. Its reliance on camera-based surveillance may also present challenges for visually impaired users.

2.4 Research Findings on Digital Accessibility in Examinations

Menger [6] conducted a comprehensive review of accessibility in online assessment, finding that the majority of online examination documents are incompatible with screen reader software and that platforms frequently require extensive manual modification before they can be usable by

visually impaired students. The review concluded that a significant accessibility gap persists in the domain of online assessment, and that existing solutions fail to provide a seamless, independent examination experience for visually impaired students.

The reviewed literature collectively confirms that while incremental progress has been made in accessible educational technology, no existing system provides a comprehensive, purpose-built, AI-integrated, voice-first examination platform specifically tailored for visually impaired students. EASVI is designed and developed to address this identified gap.

3. System Architecture and Design

3.1 Architectural Overview

EASVI is structured on a layered client-server architecture that separates concerns across four primary tiers: the Presentation Layer, the Accessibility Layer, the Application Layer, and the Data Layer. This architectural approach ensures modularity, maintainability, and scalability of the system.

- Presentation Layer: Built using HTML5, TailwindCSS, and JavaScript (ES6+), this layer renders the visual interface and manages user interactions through keyboard events.
- Accessibility Layer: Implemented via the Web Speech API, this layer intercepts system events and converts textual content into synthesized speech for delivery to the user.
- Application Layer: Powered by the Django 4.x web framework, this layer manages routing, business logic, API integrations, and session management.
- Data Layer: Implemented using MySQL or PostgreSQL, this layer provides persistent storage for user credentials, examination metadata, question data, and examination results.

3.2 Data Flow

The operational data flow of EASVI follows a structured sequence. The user accesses the web interface through a compatible browser and authenticates via keyboard input on the Login Page. The Django backend validates credentials against the database and initiates an authenticated session. Upon successful login, the Dashboard Page is presented, listing available examinations with voice announcement. When the user initiates an examination, the backend submits a structured request to the Gemini API specifying the examination subject and required question parameters. The Gemini API responds with a dynamically generated set of questions, which are

temporarily stored in the server-side session. Questions are delivered to the front-end sequentially, with the Web Speech API reading each question and its answer options aloud. Upon submission, the backend evaluates responses, computes the final score, persists the result to the database, and triggers a voice announcement of the result.

3.3 Technology Stack

Technology	Component	Purpose
HTML5	Frontend	Structure and semantic markup of web pages
TailwindCSS	Frontend	Responsive, accessible UI styling with high-contrast design
JavaScript ES6+	Frontend	Dynamic interaction, event handling, and API calls
Web Speech API	Accessibility	Text-to-speech synthesis for all voice output
Python 3.x / Django 4.x	Backend	Web framework, routing, session management, and business logic
Gemini API	AI Integration	Dynamic examination question generation
MySQL / PostgreSQL	Database	Persistent storage of users, examinations, and results

4. Key Features and Functional Modules

4.1 Voice-Based Interaction

The Web Speech API's SpeechSynthesis interface is employed throughout EASVI to deliver all examination content and system feedback as synthesized speech. Upon loading each page, the system announces the page name and prompts the user appropriately. During the examination, each question text and all associated answer options are read aloud sequentially. Navigation actions, selection confirmations, and system status messages are also announced, ensuring that the user is never dependent on visual output to understand the system's state.

4.2 Keyboard-Only Navigation

EASVI is designed for complete keyboard operability, adhering to WCAG 2.1 Success Criterion 2.1.1 (Keyboard). All interactive elements—including form fields, navigation buttons, answer options, and submission controls—are accessible via standard keyboard keys (Tab, Arrow keys, Enter, Space). Logical tab order is enforced throughout all pages, and focus indicators are visually prominent to support users with low vision.

4.3 AI-Powered Question Generation

Integration with the Gemini large language model constitutes the most technologically innovative feature of EASVI. For each examination session, the backend constructs a structured prompt specifying the examination subject, required difficulty level, and number of questions, and submits this to the Gemini API. The API returns a unique set of questions formatted for immediate use. This dynamic generation approach eliminates static question banks, ensuring each student receives a different question set, thereby significantly reducing the risk of examination fraud and improving assessment validity.

4.4 Instant Result Evaluation and Announcement

Upon submission of the examination, the Django backend immediately processes the student's responses against the correct answers returned by the Gemini API, computes the final score and percentage, and determines pass/fail status. The Result Page is rendered and the Web Speech API announces the complete result—including score, total marks, percentage, and status—within two seconds of submission. A print facility is also available, enabling the student to obtain a physical copy of their result via keyboard activation.

4.5 Accessible User Interface

All five pages of the EASVI interface—Login, Dashboard, Instructions, Examination, and Results—are built with comprehensive accessibility compliance. ARIA (Accessible Rich Internet Applications) attributes are applied to all interactive elements to ensure compatibility with external assistive technologies. High-contrast color combinations meeting WCAG AA contrast ratio requirements are used throughout. Form elements are clearly labeled, and error messages are announced via voice in addition to being displayed visually.

5. Implementation Details

5.1 Frontend Implementation

The front-end of EASVI is implemented as a multi-page web application. Each page is structured using semantic HTML5 elements (nav, main, section, article) to support logical document structure and screen reader compatibility. TailwindCSS utility classes are used to construct a responsive layout with high-contrast visual design. JavaScript event listeners are attached to keyboard events to manage navigation and interaction without mouse dependency. The Web Speech API is initialized on page load, with speech synthesis queued for all content elements.

5.2 Backend Implementation

The Django backend exposes a set of RESTful API endpoints for login, session management, examination initiation, question delivery, answer submission, and result retrieval. The Django ORM abstracts database interactions for the Users, Examinations, Questions, and Results models. Gemini API calls are executed server-side to prevent exposure of the API key in client-side code. Django's built-in session middleware manages authentication state across page transitions.

5.3 Database Design

The relational database schema includes four primary tables. The users table stores student and administrator credentials including ID, username, email address, hashed password, and role. The examinations table contains examination metadata including subject, duration, and question count. The questions table stores dynamically generated question content for each session. The results table records examination outcomes, including a foreign key reference to the user and examination tables, the score achieved, total marks, and a creation timestamp.

5.4 Hardware and Performance Requirements

Component	Specification
Processor	Intel Core i3 or equivalent (1 GHz minimum)
RAM	Minimum 4 GB
Storage	500 MB free disk space for server installation
Audio Output	Speakers or headphones (required for voice output)
Network	Stable broadband internet connection

The system targets the following performance benchmarks: Gemini API question generation within 3 seconds, speech synthesis initiation within 1 second of question loading, result computation and announcement within 2 seconds of submission, and database read/write operations completing within 500 milliseconds.

6. Comparative Analysis

A structured comparison of EASVI against existing accessible examination and educational tools clearly demonstrates the superior capabilities of the proposed system across all critical accessibility and functionality dimensions.

Feature	EASVI	JAWS	ATutor	ProctorTrack
Voice-Guided Questions	✓	Partial	✗	✗
Keyboard-Only Navigation	✓	✓	Partial	Partial
AI Question Generation	✓	✗	✗	✗
Instant Voice Result	✓	✗	✗	✗
Purpose-Built for Exam	✓	✗	✗	Partial
No External Tool Required	✓	✗	Partial	✓
ARIA Compliance	✓	N/A	Partial	Partial

The comparative analysis establishes that EASVI is the only system in the comparison that provides AI-based question generation, instant voice result announcement, and a comprehensive, purpose-built accessible examination experience without requiring any external assistive software. JAWS, while widely used, is a general-purpose screen reader that requires manual configuration for each examination platform and does not provide examination-specific features. ATutor offers basic accessibility but is not designed as an examination system. ProctorTrack focuses on invigilation rather than accessibility.

7. Results and Discussion

7.1 System Functionality

The EASVI system successfully delivers a complete examination lifecycle—from secure login and dashboard access through AI-generated question presentation, keyboard-based answer selection, and instant voice result announcement—without any requirement for visual interaction or mouse-based navigation. All five user interface pages (Login, Dashboard, Instructions, Examination, and Results) operate fully through keyboard navigation and voice feedback, as designed.

7.2 Accessibility Compliance

The system meets the key accessibility requirements specified in WCAG 2.1, including keyboard operability (Success Criterion 2.1.1), text alternatives for non-text content, sufficient color contrast ratios, and the use of ARIA attributes for all interactive components. Accessibility testing using

browser developer tools and screen reader simulation confirmed accurate voice announcements and functional keyboard navigation paths across all pages.

7.3 AI Question Generation

The Gemini API integration was validated across multiple examination subjects, consistently generating contextually appropriate, unique, and well-formed examination questions within the specified 3-second performance target under standard network conditions. The dynamic generation mechanism effectively eliminates the limitations of static question banks and demonstrably enhances examination integrity.

7.4 Performance

System performance testing confirmed that result computation and voice announcement occurred within 2 seconds of submission in all tested scenarios. Question delivery and speech synthesis initiation consistently met the 1-second target. Database operations completed within the 500-millisecond benchmark under standard load conditions.

8. Future Scope

The current implementation of EASVI provides a strong foundation for future enhancement. Several directions for development are identified:

- Speech-to-Answer Input: Incorporating speech recognition via the Web Speech API's SpeechRecognition interface to allow students to answer questions verbally, further reducing physical interaction requirements.
- Mobile Application: Developing native Android and iOS applications to expand accessibility to smartphone and tablet platforms, which are increasingly primary computing devices in developing regions.
- Multi-Language Support: Integrating multi-lingual question generation and voice output to serve visually impaired students across diverse linguistic communities in India and internationally.
- Adaptive Assessment: Implementing AI-driven adaptive questioning, where question difficulty adjusts dynamically based on student performance, providing more accurate and personalized assessments.
- Accessible Proctoring Integration: Incorporating non-visual proctoring solutions to maintain examination integrity for remote assessments without relying on visual surveillance.

- Offline Functionality: Implementing service worker-based offline capabilities to support students in regions with unreliable internet connectivity.
- Educator Analytics Dashboard: Providing faculty and administrators with a dedicated analytics interface for reviewing student performance, identifying learning gaps, and monitoring examination trends.

9. Conclusion

The Examination System for Visually Impaired (EASVI) demonstrates that it is technically feasible and practically achievable to build a fully accessible, self-reliant online examination platform using modern, freely available web technologies, without reliance on proprietary assistive software. The system successfully integrates the Web Speech API for voice-based interaction, keyboard-only navigation for independent operability, and the Gemini AI model for dynamic question generation into a cohesive and functional examination platform.

EASVI represents a significant advancement over existing accessible examination solutions by delivering the complete examination lifecycle—from login to result announcement—through a voice-first, AI-powered interface. The system directly addresses the identified deficiencies of current tools, which lack AI question generation, provide limited voice integration, and are not purpose-built for the examination context. By empowering visually impaired students to independently demonstrate their academic potential, EASVI contributes meaningfully to the goals of inclusive education and digital accessibility, and aligns with the WCAG 2.1 standards and the UN Sustainable Development Goal 4 (Quality Education).

EASVI can serve as a replicable and extensible model for academic institutions and professional certification bodies seeking to provide equitable digital assessment experiences for students with visual disabilities. Future iterations incorporating speech-to-answer input, mobile platform support, and adaptive assessment capabilities have the potential to further elevate the system's impact and reach.

References

[1] W3C Web Accessibility Initiative, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[2] Google LLC, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/docs

[3] Mozilla Developer Network, "Web Speech API," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[4] Django Software Foundation, "Django Documentation," Django Project, 2024. [Online]. Available: https://docs.djangoproject.com/

[5] A. Yesilada, S. Harper, C. Goble, and R. Stevens, "Screen Readers Cannot See," in Proc. International Conference on Web Engineering, 2004, pp. 445–458.

[6] P. Menger, "Accessibility in Online Assessment: A Review of Current Practice and Future Directions," Journal of Inclusive Education Technology, vol. 12, no. 3, pp. 45–62, 2022.

[7] Freedom Scientific, "JAWS Screen Reader," 2024. [Online]. Available: https://www.freedomscientific.com/products/software/jaws/

[8] ATutor, "ATutor Learning Management System," 2024. [Online]. Available: https://atutor.github.io/

[9] Verificient Technologies, "ProctorTrack Remote Proctoring Solution," 2024. [Online]. Available: https://www.verificient.com/proctortrack/

[10] TailwindCSS, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs